

Evolutionary Architecture for a Car Leasing Domain — The Full Journey

Author: Future Me

Context: Learning evolutionary architecture by transforming a car leasing application through 3 architectural stages

Repository: [evolutionary-architecture-csharp](#)

Domain: Car Leasing vs. Buying decision platform

Tech Stack: .NET 10, ASP.NET Core, EF Core, PostgreSQL, MassTransit, RabbitMQ, MediatR, Dapper

Table of Contents

1. [Prologue — Why This Journey Exists](#)
 2. [Chapter 1 — The Monolith: Getting Something Working](#)
 3. [Chapter 2 — Modular Monolith: Adding Structure Before It Hurts](#)
 4. [Chapter 3 — Microservice Extraction: Splitting When Cohesion Demands It](#)
 5. [The Leasing Calculation Engine](#)
 6. [Technology Deep Dive — Every Fix, Every Surprise](#)
 7. [Architecture Decision Records](#)
 8. [Epilogue — Lessons for Future Me](#)
 9. [Testing Strategy](#)
-

1. Prologue — Why This Journey Exists

1.1 The Original Project

The reference repository, `evolutionary-architecture-by-example`, demonstrates how to evolve software architecture incrementally rather than doing a big-bang rewrite. Each chapter represents a real evolutionary step:

- **Chapter 1:** A single-project monolith — namespaces as the only separation between domain concepts
- **Chapter 2:** A modular monolith — each domain concept gets its own project, database schema, and explicit dependencies
- **Chapter 3:** Microservice extraction — one module becomes a standalone service, communicating via async messaging over RabbitMQ

The original domain is a fitness studio (passes, contracts, offers, reports). I chose to adapt it to a **car leasing domain** (Lease vs. Buy) to make it personally relevant and to force myself to truly understand the architecture rather than mindlessly copying code.

The reference repository uses .NET 7. Our implementation uses .NET 10. This version gap (7 → 10) introduced several breaking changes that required careful adaptation — see the Technology Deep Dive section for every single one.

1.2 The Leasing Domain

The application helps users compare the total cost of leasing versus buying a vehicle:

- **Leasing** — A customer prepares a lease agreement, subject to business rules (minimum credit score of 700, maximum annual mileage of 15,000, maximum term of 36 months). Once prepared, the lease

can be signed within 14 days. Signing publishes a `LeaseSignedEvent` that other modules react to.

- **Purchasing** — A customer gets a financing offer (down payment, APR, term). The monthly payment is calculated using the standard loan amortization formula. When the loan is fully paid off, a `PurchaseCompletedEvent` is published.
- **Vehicles** — A catalog of vehicles with their MSRP, residual values, and current status (Available, Leased, Owned). The Vehicles module reacts to both `LeaseSignedEvent` and `PurchaseCompletedEvent` to update vehicle status.
- **Comparison** — A placeholder module that will eventually generate cost comparison reports between leasing and buying. Uses Dapper for read-only queries.

1.3 The Evolutionary Path

Chapter 1 (leasebuyarch/)	Chapter 2 (leasebuyarch2/)	Chapter 3 (leasebuyarch3/)
Single .csproj	Multiple .csproj	3 solutions (25 projects)
All code in /Src	Module-per-project	Common/ (shared packages)
Namespace partitioning	Clean Architecture per module	Leasing/ (microservice)
No DI container (remaining)		ModularMonolith/
No EF Core	Shared kernel (Common)	MassTransit/RabbitMQ
No event bus	In-Memory event bus	Event bus becomes out-of-process
No feature flags	(MediatR)	Feature flags via
IFeatureManager	Feature flags via IConfiguration	

1.4 Why "Evolutionary" Architecture?

The key insight of evolutionary architecture is that **you don't design the final architecture upfront**. Instead, you start simple and let the architecture evolve as your understanding of the domain deepens and as constraints change.

In practice, this means:

1. **Start with a monolith** (Chapter 1) because it's the fastest way to validate the domain model. You cannot modularize what you do not understand.
2. **Introduce modular boundaries** (Chapter 2) when the monolith becomes hard to change. Team coordination overhead, testing difficulty, and deployment risk are signals that modularization is needed.
3. **Extract microservices** (Chapter 3) when a module has different scalability, deployability, or team ownership requirements than the rest of the system.

The beauty of this approach is that **each chapter is a working system**. You can ship Chapter 1 to production. You can ship Chapter 2 to production. Each evolutionary step is independently valuable.

1.5 The Fitness Studio to Leasing Domain Mapping

To understand how the adaptation worked, here is the exact mapping:

Fitness Studio (Reference)	Car Leasing (Our Implementation)
Contract	Lease

PrepareContractCommand	PrepareLeaseCommand
ContractSignedEvent	LeaseSignedEvent
Pass (membership)	Purchase (financing)
PassExpiredEvent	PurchaseCompletedEvent
Reports	Comparison
Offers	Vehicles (residual values)

The mapping is not one-to-one in all cases. The Vehicles module, for example, was inspired by the Offers module in the reference but adapted to track vehicle inventory status. The Comparison module is a placeholder that maps to the Reports module.

2. Chapter 1 — The Monolith: Getting Something Working

2.1 Goal

Build the entire application in a single .NET project. No separation by projects, no fancy patterns. Just namespaces to organize code. The point is to **defer architectural decisions** until you understand the domain.

2.2 Project Structure

```
leasebuyarch/
├── Src/
│   ├── LeaseBuyArch.sln
│   ├── LeaseBuyArch.csproj
│   ├── Program.cs                # All Minimal API endpoints
│   ├── appsettings.json
│   ├── appsettings.Development.json
│   ├── docker-compose.yml
│   ├── Models/                  # Domain models
│   │   ├── Lease.cs
│   │   ├── Purchase.cs
│   │   └── Vehicle.cs
│   ├── Services/                # Business logic
│   │   ├── LeasingService.cs
│   │   ├── PurchasingService.cs
│   │   └── ComparisonService.cs
│   ├── Data/                    # EF Core
│   │   ├── AppDbContext.cs
│   │   └── Migrations/
│   └── Makefile
```

2.3 Domain Models as Plain Old CLR Objects

In Chapter 1, domain models were simple data containers with some logic:

```

public class Lease
{
    public Guid Id { get; set; }
    public Guid CustomerId { get; set; }
    public decimal MonthlyPayment { get; set; }
    public DateTimeOffset PreparedAt { get; set; }
    public DateTimeOffset? SignedAt { get; set; }
    public bool Signed => SignedAt.HasValue;

    // Business logic is in the model itself
    public void Sign(DateTimeOffset signedAt)
    {
        if ((signedAt - PreparedAt).TotalDays > 14)
            throw new InvalidOperationException("Lease can only be signed within 14
days");
        SignedAt = signedAt;
    }
}

```

This works for Chapter 1. The problem is that business rules are embedded in model methods with no way to reuse, compose, or test them independently. Later chapters extract these into dedicated BusinessRule classes.

2.4 Endpoints Defined Inline in Program.cs

Chapter 1 used the simplest possible endpoint structure — everything in `Program.cs` :

```

app.MapPost("/api/leasing", async (PrepareLeaseRequest request, AppDbContext db) =>
{
    var lease = new Lease
    {
        Id = Guid.NewGuid(),
        CustomerId = request.CustomerId,
        // ... more properties
        PreparedAt = DateTimeOffset.UtcNow
    };

    // Business rule validation inline
    if (request.CreditScore < 700)
        return Results.Conflict("Credit score too low");
    if (request.AnnualMileageLimit > 15000)
        return Results.Conflict("Mileage exceeds maximum");

    db.Leases.Add(lease);
    await db.SaveChangesAsync();
    return Results.Created($"/api/leasing/{lease.Id}", lease.Id);
});

```

Problems with this approach:

- Endpoints are not discoverable (everything is in one file)

- Business rules are mixed with HTTP concerns
- No separation of concerns — the endpoint knows about the database, validation, and response formatting
- Testing requires spinning up the full web application

2.5 Dependency Injection Setup

Even Chapter 1 used DI:

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));
```

This is the minimum viable DI setup. No MediatR, no event bus, no feature flags.

2.6 Key Decisions

Decision	Rationale
Single project	Zero ceremony. You can run <code>dotnet new</code> and start coding.
Namespace modules	<code>LeaseBuyArch.Models.Leasing</code> , <code>LeaseBuyArch.Services.Purchasing</code> — purely organizational
EF Core directly in endpoints	No CQRS, no MediatR. Endpoints call <code>dbContext.Leases.AddAsync(...)</code>
PostgreSQL	Same as reference. Good choice for modular separation later (database schemas)
docker-compose with postgres	So the app actually runs. No in-memory database fakes.

2.7 What the Monolith Taught Me

1. **Domain discovery happens in the monolith.** During Chapter 1, I changed the leasing calculation formula three times as I learned about money factors, residual values, and depreciation. A modular architecture would have made these changes more expensive.
2. **Namespace boundaries are social, not technical.** Nothing prevents `LeasingService` from calling `PurchasingService` or directly accessing `Vehicles` tables. This is fine for a team of one, but becomes a problem as the team grows.
3. **The cost of refactoring is low when there's no structure.** Moving a class from one namespace to another is trivial. Moving a class from one project to another requires `csproj` changes, assembly reference updates, and namespace import adjustments.
4. **dotnet new defaults matter.** .NET 10 enables `<ImplicitUsings>enable</ImplicitUsings>` and `<Nullable>enable</Nullable>` by default. Nullable reference types caught several potential null reference bugs during development. However, they also required careful handling of `string` properties — should they be `string` (non-nullable), `string?` (nullable), or initialized to `string.Empty`?

2.8 Technology Fixes in Chapter 1

Problem: Npgsql.EntityFrameworkCore.PostgreSQL version compatibility.

Fix: Always check the NuGet page for the correct EF Core + Npgsql version matrix. At the time, EF Core 10.0.8 required Npgsql.EntityFrameworkCore.PostgreSQL 10.0.1.

Problem: `TreatWarningsAsErrors` caused NU1510 (package pruning warnings) to fail the build.

Fix: When you add `<FrameworkReference Include="Microsoft.AspNetCore.App" />`, many NuGet packages like `Microsoft.Extensions.DependencyInjection.Abstractions` become duplicate dependencies — they are already included in the ASP.NET Core shared framework. The .NET SDK 10 prunes them and warns. Either remove the explicit package reference or suppress the warning with `<NoWarn>$(NoWarn);NU1510</NoWarn>`.

Problem: OpenAPI operation descriptions using `WithOpenApi()` were deprecated in .NET 10.

Fix: Use `AddOpenApiOperationTransformer((operation, context, ct) => { ... })` instead. This is the .NET 10 pattern for customizing OpenAPI operations. The old pattern accepted and returned an `OpenApiOperation` object; the new pattern mutates the operation in place and returns `Task`.

```
// Old (.NET 7-9):
app.MapPost("/api/leasing", ...)
    .WithOpenApi(operation =>
    {
        operation.Summary = "Prepares a new lease";
        return operation;
    });

// New (.NET 10):
app.MapPost("/api/leasing", ...)
    .AddOpenApiOperationTransformer((operation, context, ct) =>
    {
        operation.Summary = "Prepares a new lease";
        return Task.CompletedTask;
    });
```

2.9 Additional Chapter 1 Discoveries

2.9.1 FluentValidation Integration

Chapter 1 uses `FluentValidation` for request validation, a pattern carried through all three chapters:

```
internal sealed class PrepareLeaseRequestValidator :
    AbstractValidator<PrepareLeaseRequest>
{
    public PrepareLeaseRequestValidator()
    {
        RuleFor(request => request.CustomerId).NotEmpty();
        RuleFor(request => request.VehicleId).NotEmpty();
        RuleFor(request => request.VehicleMsrp).GreaterThan(0);
        RuleFor(request => request.ResidualPercentage).InclusiveBetween(1, 99);
        RuleFor(request => request.MoneyFactor).GreaterThan(0);
        RuleFor(request => request.TermMonths).InclusiveBetween(12, 60);
        RuleFor(request => request.AnnualMileageLimit).GreaterThan(0);
    }
}
```

```

        RuleFor(request => request.CreditScore).InclusiveBetween(300, 850);
    }
}

```

Validators are registered via `AddRequestsValidations()` which uses assembly scanning from `FluentValidation.DependencyInjectionExtensions`. The `ValidateRequest<T>()` extension method is called on endpoints to auto-validate before the handler executes:

```

app.MapPost(LeasingApiPaths.Prepare, async (...) => { ... })
    .ValidateRequest<PrepareLeaseRequest>()

```

2.9.2 The Empty SignLeaseRequest

The `SignLeaseRequest` is an empty record — the endpoint only needs the lease ID from the URL:

```

public sealed record SignLeaseRequest();

```

Its validator is similarly empty:

```

internal sealed class SignLeaseRequestValidator :
    AbstractValidator<SignLeaseRequest>
{
    public SignLeaseRequestValidator()
    {
    }
}

```

This is valid `FluentValidation` — a validator with no rules passes all requests. It exists because the infrastructure always validates when `ValidateRequest<T>()` is applied.

2.9.3 The API Paths Constant Pattern

Chapter 1 introduces the `ApiPaths` pattern used throughout:

```

public static class ApiPaths
{
    public const string Root = "api";
}

internal static class LeasingApiPaths
{
    private const string LeasingRootApi = $"{ApiPaths.Root}/leasing";
    internal const string Prepare = LeasingRootApi;
    internal const string Sign = $"{LeasingRootApi}/{id}";
}

```

This ensures all endpoints are under `/api/` and makes refactoring trivial — to move all endpoints to `/api/v1/`, change one constant.

2.9.4 The Empty Module Placeholder

Chapter 1 has empty module classes that serve as placeholders:

```
internal static class OfferPurchaseModule
{
}
```

These exist because the reference architecture creates a Module class for every bounded context action, even if initialization isn't needed. They're placeholders for future logic.

2.9.5 Response Type Documentation

Endpoints explicitly document their response types for OpenAPI:

```
.Produces<string>(StatusCodes.Status201Created)
.Produces(StatusCodes.Status409Conflict)
.Produces(StatusCodes.Status500InternalServerError)
```

Without these, Swagger shows "No response" for each endpoint. Chapter 2/3 omitted these — a minor regression in API documentation.

3. Chapter 2 — The Modular Monolith: Adding Structure Before It Hurts

3.1 Goal

Split the monolith into separate projects per module while keeping everything in a single process. Each module gets:

- Its own project (or projects, following Clean Architecture layers)
- Its own database schema (enforces physical separation in the database)
- Its own set of NuGet and project dependencies
- An in-memory event bus for cross-module communication
- Feature flags to enable/disable modules independently

3.2 Why Modular Monolith?

A modular monolith is a single process with multiple modules that communicate through well-defined interfaces. It offers:

- **Team scalability** — Different teams can work on different modules without merge conflicts
- **Testing isolation** — Modules can be tested independently
- **Deployment simplicity** — Still a single binary, easy to deploy
- **Evolution path** — Any module can be extracted into a microservice later (which is exactly what Chapter 3 does)

Modular monolith is NOT the same as a "structured monolith." A structured monolith has separate projects but allows uncontrolled cross-project dependencies. A true modular monolith enforces boundaries — no circular dependencies, no cross-module direct calls, only event-based communication.

3.3 Project Structure (15 projects)

```
leasebuyarch2/Src/
├── Directory.Build.props          # Shared build settings for all 15 projects
```



```

├─ LeaseBuyArch.slnx          # Solution file (new .slnx format)
├─ Dockerfile
├─ docker-compose.yml
├─ Makefile
├─
├─ Common/                   # Shared kernel (3 projects)
│   ├─ LeaseBuyArch.Common.Core/    # BusinessRules, SystemClock
│   ├─ LeaseBuyArch.Common.Api/     # ApiPaths, ErrorHandling
│   └─ LeaseBuyArch.Common.Infrastructure/ # EventBus, Mediator, Modules
├─
├─ Leasing/                  # Full Clean Architecture (5 projects)
│   ├─ LeaseBuyArch.Leasing.Core/    # Domain entities, business rules
│   ├─ LeaseBuyArch.Leasing.IntegrationEvents/ # LeaseSignedEvent
│   ├─ LeaseBuyArch.Leasing.Application/ # Commands, handlers
│   ├─ LeaseBuyArch.Leasing.Infrastructure/ # EF, repos, MediatR setup
│   └─ LeaseBuyArch.Leasing.Api/      # Endpoints, module registration
├─
├─ Purchasing/               # Simpler 3-project structure
│   ├─ LeaseBuyArch.Purchasing.DataAccess/ # EF, Purchase entity
│   ├─ LeaseBuyArch.Purchasing.IntegrationEvents/ # PurchaseCompletedEvent
│   └─ LeaseBuyArch.Purchasing.Api/      # Endpoints, module registration
├─
├─ Vehicles/                 # 2-project structure (no events project)
│   ├─ LeaseBuyArch.Vehicles.DataAccess/ # EF, Vehicle entity
│   └─ LeaseBuyArch.Vehicles.Api/        # Endpoints, event handlers
├─
├─ Comparison/               # Single project (Dapper-based)
│   └─ LeaseBuyArch.Comparison/          # Endpoints, Dapper queries
├─
└─ LeaseBuyArch/              # Host project (composition root)
    ├─ Program.cs
    ├─ Module.cs
    ├─ appsettings.json
    ├─ appsettings.Development.json
    └─ Properties/launchSettings.json

```

3.4 The Shared Kernel (Common Layer)

The shared kernel is the **minimum surface area** that all modules depend on. Think of it as a "micro-framework" for the application.

3.4.1 Common.Core — Business Rules and System Clock

Business Rules Pattern:

```

public interface IBusinessRule
{
    bool IsMet();
    string Error { get; }
}

public static class BusinessRuleValidator

```

```

{
    public static void Validate(IBusinessRule rule)
    {
        if (!rule.IsMet())
            throw new BusinessException(rule.Error);
    }
}

public class BusinessException : InvalidOperationException
{
    public BusinessException(string message) : base(message) { }
}

```

Each business rule is a separate class:

```

internal sealed class CustomerCreditScoreMustBeHighEnoughRule : IBusinessRule
{
    private readonly int _creditScore;
    private readonly int _minimum;

    internal CustomerCreditScoreMustBeHighEnoughRule(int creditScore, int minimum)
    {
        _creditScore = creditScore;
        _minimum = minimum;
    }

    public bool IsMet() => _creditScore >= _minimum;
    public string Error => $"Customer credit score {_creditScore} is below minimum {_minimum}";
}

```

Why separate rule classes? Each rule can be unit tested independently. Rules can be composed. The error message is consistent and descriptive. The pattern is extensible — adding a new rule means adding a new class without modifying existing code (Open/Closed Principle).

System Clock Abstraction:

```

public interface ISystemClock
{
    DateTimeOffset Now { get; }
}

internal sealed class SystemClock : ISystemClock
{
    public DateTimeOffset Now => DateTimeOffset.UtcNow;
}

public static class SystemClockModule
{
    public static IServiceCollection AddSystemClock(this IServiceCollection
services)

```

```

    {
        services.AddSingleton<ISystemClock, SystemClock>();
        return services;
    }
}

```

Why abstract the system clock? Without this, tests that depend on the current time are flaky — they pass at one time of day but fail at another. With `ISystemClock`, tests can inject a fake clock that returns a fixed date.

3.4.2 Common.Api — Error Handling and API Conventions

Exception Middleware:

```

internal sealed class ExceptionMiddleware
{
    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            await HandleExceptionAsync(context, ex);
        }
    }

    private static async Task HandleExceptionAsync(HttpContext context, Exception
exception)
    {
        context.Response.ContentType = "application/json";
        int statusCode;
        string message;

        switch (exception)
        {
            case BusinessException:
                statusCode = 409; // Conflict
                message = exception.Message;
                break;
            case ResourceNotFoundException:
                statusCode = 404;
                message = exception.Message;
                break;
            default:
                statusCode = 500;
                message = exception.Message;
                break;
        }

        context.Response.StatusCode = statusCode;
    }
}

```

```

        var result = JsonSerializer.Serialize(new
ExceptionResponseMessage(statusCode, message));
        await context.Response.WriteAsync(result);
    }
}

```

Why 409 for business rule violations? HTTP 409 Conflict means "the request conflicts with the current state of the resource." When a business rule prevents an operation, it's a semantic conflict — the request is valid but can't be fulfilled given the current state. 400 Bad Request would mean the request itself is malformed, which is not the case here.

3.4.3 Common.Infrastructure — Event Bus, Mediator, Module Management

Integration Event Interfaces:

```

// Extends MediatR's INotification so handlers are discovered automatically
public interface IIntegrationEvent : INotification
{
    Guid Id { get; }
    DateTimeOffset OccurredDateTime { get; }
}

// Extends MediatR's INotificationHandler<T>
public interface IIntegrationEventHandler<in TEvent> : INotificationHandler<TEvent>
    where TEvent : IIntegrationEvent
{
}

```

In-Memory Event Bus:

```

public sealed class InMemoryEventBus : IEventBus
{
    private readonly IMediator _mediator;

    public InMemoryEventBus(IMediator mediator) => _mediator = mediator;

    public async Task PublishAsync<TEvent>(TEvent @event, CancellationToken ct =
default)
        where TEvent : IIntegrationEvent =>
        await _mediator.Publish(@event, ct);
}

```

The `InMemoryEventBus` wraps MediatR's `IMediator.Publish()`. When an event is published, MediatR dispatches it synchronously to all registered `INotificationHandler<T>` implementations. This means handlers run **in the same process and on the same thread** as the publisher (unless the handler explicitly starts a new task).

Module Availability Checker:

```

public static class ModuleAvailabilityChecker
{

```

```

    public static bool IsModuleEnabled(this IConfiguration configuration, string
module)
    {
        var featureManagement = configuration.GetSection("FeatureManagement");
        return featureManagement.GetValue<bool>(module);
    }

    public static bool IsModuleEnabled(this IApplicationBuilder app, string module)
    {
        var configuration =
app.ApplicationServices.GetService<IConfiguration>();
        return configuration.IsModuleEnabled(module);
    }
}

```

This checks `appsettings.json` :

```

{
  "FeatureManagement": {
    "Leasing": true,
    "Purchasing": true,
    "Vehicles": true,
    "Comparison": true
  }
}

```

3.5 Clean Architecture Inside the Leasing Module

The Leasing module follows Clean Architecture strictly:

LeaseBuyArch.Leasing.Core	→ Innermost: entities, business rules
LeaseBuyArch.Leasing.Application	→ Use cases: commands, handlers
LeaseBuyArch.Leasing.Infrastructure	→ External concerns: EF, repositories
LeaseBuyArch.Leasing.Api	→ Interface adapters: endpoints, module registration
LeaseBuyArch.Leasing.IntegrationEvents	→ Shared events (cross-module contract)

Dependency rule: Dependencies point inward. Core has no dependencies. Application depends on Core. Infrastructure depends on Application. Api depends on Infrastructure.

Why this matters: The Leasing module can be extracted into a microservice (Chapter 3) without modifying any business logic. The Core and Application projects move verbatim. Only the Infrastructure (EF configuration, connection strings) and Api (endpoint routing) need adjustment.

3.5.1 The Command Pattern

Commands encapsulate the data needed for an operation:

```

public sealed record PrepareLeaseCommand(
    Guid CustomerId, Guid VehicleId, decimal VehicleMsrp,

```

```
decimal ResidualPercentage, decimal MoneyFactor, int TermMonths,  
int AnnualMileageLimit, int CreditScore) : ICommand<Guid>;
```

Notice `ICommand<Guid>` — the command returns the new Lease ID. For commands that don't return a value, `ICommand` (without type parameter) is used.

3.5.2 The Command Handler

```
internal sealed class PrepareLeaseCommandHandler :  
IRequestHandler<PrepareLeaseCommand, Guid>  
{  
    private readonly ILeasingRepository _repository;  
    private readonly ISystemClock _systemClock;  
  
    public PrepareLeaseCommandHandler(ILeasingRepository repository, ISystemClock  
systemClock)  
    {  
        _repository = repository;  
        _systemClock = systemClock;  
    }  
  
    public async Task<Guid> Handle(PrepareLeaseCommand command, CancellationToken  
cancellation_token)  
    {  
        var previousLease = await _repository.GetPreviousForCustomerAsync(  
            command.CustomerId, cancellation_token);  
        var lease = Lease.Prepare(  
            command.CustomerId, command.VehicleId, command.VehicleMsrp,  
            command.ResidualPercentage, command.MoneyFactor, command.TermMonths,  
            command.AnnualMileageLimit, command.CreditScore, _systemClock.Now,  
            previousLease?.Signed);  
        await _repository.AddAsync(lease, cancellation_token);  
        await _repository.SaveChangesAsync(cancellation_token);  
        return lease.Id;  
    }  
}
```

What happens here:

1. The handler receives the command via MediatR
2. It loads the previous lease for the same customer (to check if it was settled)
3. It calls the static factory method `Lease.Prepare(...)` which validates all business rules
4. It persists the new lease
5. It returns the new lease ID

The handler is thin — it orchestrates but doesn't contain business logic. All business rules are in the `Lease` entity.

3.5.3 The Sign Command with Event Publishing

```

internal sealed class SignLeaseCommandHandler : IRequestHandler<SignLeaseCommand>
{
    private readonly ILeasingRepository _repository;
    private readonly ISystemClock _systemClock;
    private readonly IEventBus _eventBus;

    public async Task Handle(SignLeaseCommand command, CancellationToken
cancellationTokens)
    {
        var lease = await _repository.GetByIdAsync(command.Id, cancellationTokens)
            ?? throw new InvalidOperationException($"Lease with id {command.Id} not
found");

        lease.Sign(_systemClock.Now);
        await _repository.SaveChangesAsync(cancellationTokens);

        var @event = LeaseSignedEvent.Create(lease.Id, lease.CustomerId,
lease.VehicleId,
            lease.MonthlyPayment, lease.TermMonths, lease.SignedAt!.Value);
        await _eventBus.PublishAsync(@event, cancellationTokens);
    }
}

```

The handler does three things:

1. Applies business logic (signing the lease)
2. Persists the change
3. Publishes an integration event for other modules to consume

The order matters. The event is published AFTER the database transaction commits. If the event were published before, a consumer might read stale data. This is a simple form of the "outbox pattern."

3.6 Simpler Modules (Purchasing, Vehicles, Comparison)

Not all modules need Clean Architecture. The Purchasing module, for example, is simpler because its business logic is minimal:

```

Purchasing/
├─ LeaseBuyArch.Purchasing.DataAccess/      # EF DbContext, Purchase entity
├─ LeaseBuyArch.Purchasing.IntegrationEvents/ # PurchaseCompletedEvent
└─ LeaseBuyArch.Purchasing.Api/              # Endpoints (no Application layer)

```

Endpoints inject EF DbContext directly:

```

app.MapPatch(PurchasingApiPaths.Complete, async (Guid id, PurchasingPersistence
persistence,
    IEventBus bus, ISystemClock systemClock, CancellationToken ct) =>
{
    var purchase = await persistence.Purchases.FindAsync(new object?[] { id },
ct);

    if (purchase is null) return Results.NotFound();
    purchase.Complete(systemClock.Now);
}

```

```

        await persistence.SaveChangesAsync(ct);
        var @event = PurchaseCompletedEvent.Create(purchase.Id, purchase.CustomerId,
            purchase.VehicleId, purchase.CompletedAt!.Value);
        await bus.PublishAsync(@event, ct);
        return Results.NoContent();
    });

```

Why is this OK? Because the Purchasing module's complexity is low — it has one entity, two endpoints, and simple business logic. Adding a full Application layer with commands and handlers would be over-engineering. The Leasing module, with complex business rules (credit score validation, mileage checks, term limits, previous lease checks, 14-day signing window), justifies the extra structure.

3.7 Initializing the Database

Each module that uses EF Core has its own migrations and applies them at startup:

```

public static WebApplication UseDatabase(this WebApplication app)
{
    using var scope = app.Services.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<LeasingPersistence>();
    context.Database.Migrate(); // Applies pending migrations
    return app;
}

```

Why Migrate() and not EnsureCreated() ? `EnsureCreated()` creates the database but does not track changes — subsequent runs won't apply new migrations. `Migrate()` applies pending migrations and tracks which have been applied.

Risk: Calling `Migrate()` at startup means the application needs database write permissions. In production, database migrations are typically run as a separate step in the CI/CD pipeline.

3.8 The Module Record

```

internal record Module(string Value)
{
    internal static readonly Module Leasing = new("Leasing");
    internal static readonly Module Purchasing = new("Purchasing");
    internal static readonly Module Vehicles = new("Vehicles");
    internal static readonly Module Comparison = new("Comparison");

    public static implicit operator string(Module module) => module.Value;
}

```

Why internal ? The Module record is an implementation detail of the host project. Module names are used for feature flag lookups and route registration, but no external code should depend on them.

Why implicit operator string ? This allows passing `Module.Purchasing` directly to methods expecting `string` :

```

services.AddPurchasing(configuration, Module.Purchasing);
//

```



```
// implicitly converts to "Purchasing"
```

3.9 The Module Registration Pattern

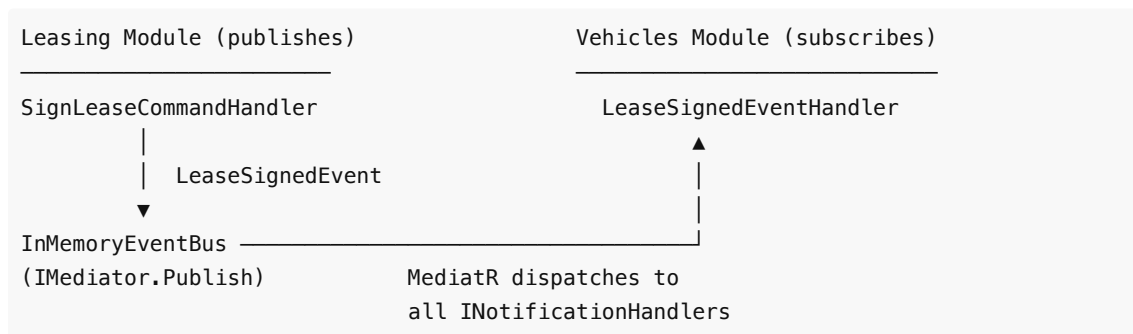
Every module follows the same two-phase pattern:

```
// Phase 1: Service registration (called in Program.cs, before Build)
public static IServiceCollection AddPurchasing(this IServiceCollection services,
    IConfiguration configuration, string module)
{
    if (!configuration.IsModuleEnabled(module)) return services; // Feature flag
    gate
    services.AddDatabase(configuration);
    return services;
}

// Phase 2: Endpoint mapping (called in Program.cs, after Build)
public static WebApplication RegisterPurchasing(this WebApplication app, string
module)
{
    if (!app.IsModuleEnabled(module)) return app; // Feature flag gate
    app.UseDatabase();
    app.MapPurchasing();
    return app;
}
```

Why two phases? In ASP.NET Core, service registration (`Add*`) happens before `builder.Build()` . Endpoint mapping happens after `builder.Build()` . This separation is enforced by the framework — you cannot add services after `Build()` .

3.10 Cross-Module Communication Flow



And in the handler:

```
internal sealed class LeaseSignedEventHandler :
    IIntegrationEventHandler<LeaseSignedEvent>
{
    private readonly VehiclesPersistence _persistence;

    public async Task Handle(LeaseSignedEvent @event, CancellationToken ct)
```

```

    {
        var vehicle = await _persistence.Vehicles.FindAsync(
            new object?[] { @event.VehicleId }, ct);
        if (vehicle is null) return;
        vehicle.MarkAsLeased();
        await _persistence.SaveChangesAsync(ct);
    }
}

```

Important: The handler is in the Vehicles module, not in the Leasing module. The Leasing module only publishes the event. It does not know (or care) who consumes it. This is the essence of event-driven architecture — **publishers and subscribers are decoupled**.

3.11 Directory.Build.props — The Unsung Hero

One `Directory.Build.props` at the solution root applies to all projects:

```

<Project>
  <PropertyGroup>
    <AssemblyName>EvolutionaryArchitecture.${MSBuildProjectName}</AssemblyName>
    <RootNamespace>$(AssemblyName)</RootNamespace>
    <TargetFramework>net10.0</TargetFramework>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>
  </PropertyGroup>
  <ItemGroup>
    <InternalsVisibleTo Include="DynamicProxyGenAssembly2" />
  </ItemGroup>
  <ItemGroup>
    <Using Include="Microsoft.AspNetCore.Builder" />
    <Using Include="Microsoft.AspNetCore.Http" />
    <Using Include="Microsoft.AspNetCore.Routing" />
    <Using Include="Microsoft.Extensions.Configuration" />
    <Using Include="Microsoft.Extensions.DependencyInjection" />
  </ItemGroup>
</Project>

```

What this does:

- `AssemblyName` : Every project gets the `EvolutionaryArchitecture.` prefix automatically
- `TargetFramework` : All projects target .NET 10.0
- `TreatWarningsAsErrors` : No warnings allowed (caught us many times!)
- `ImplicitUsings` : Global `using` statements for `System`, `System.Collections.Generic`, etc.
- `Nullable` : Nullable reference types enabled
- `InternalsVisibleTo` : Moq can mock internal types
- `Global Usings` : Every file has access to `IApplicationBuilder` , `HttpContext` , `IConfiguration` , `IServiceCollection` , etc.

This single file eliminates hundreds of `using` directives across 15 projects.

3.12 The .slnx Format

Starting in .NET 10, the new `.slnx` XML-based solution format replaces the old `.sln` binary format:

```
<Solution>
  <Folder Name="/Common/">
    <Project Path="Common/LeaseBuyArch.Common.Api/LeaseBuyArch.Common.Api.csproj" />
  </Folder>
  <Folder Name="/Leasing/">
    <Project Path="Leasing/LeaseBuyArch.Leasing.Api/LeaseBuyArch.Leasing.Api.csproj" />
    <Project Path="Leasing/LeaseBuyArch.Leasing.Application/LeaseBuyArch.Leasing.Application.csproj" />
    <Project Path="Leasing/LeaseBuyArch.Leasing.Core/LeaseBuyArch.Leasing.Core.csproj" />
    <Project Path="Leasing/LeaseBuyArch.Leasing.Infrastructure/LeaseBuyArch.Leasing.Infrastructure.csproj" />
    <Project Path="Leasing/LeaseBuyArch.Leasing.IntegrationEvents/LeaseBuyArch.Leasing.IntegrationEvents.csproj" />
  </Folder>
  ...
</Solution>
```

Benefits of .slnx over .sln:

- XML format is merge-friendly (old `.sln` format has binary GUIDs that conflict in merges)
- Supports solution folders natively
- Easy to generate programmatically
- No more GUIDs — projects are referenced by path

3.13 Technology Fixes in Chapter 2

Problem: `Module` record defined in TWO places (`Common.Core` and the host project).

Fix: Remove the duplicate. The `Module` record belongs in the **host project** because it's the composition root — the place where modules are wired together. `Common.Core` should not know about specific module names. That would create a coupling where adding a module requires modifying the shared kernel.

Problem: The `ModuleAvailabilityChecker` used

`IConfiguration.GetSection("FeatureManagement").GetValue<bool>(module)` in some places and `IConfiguration.GetSection("FeatureManagement").Get<bool>(module)` in others. The latter doesn't exist — `Get<T>` is on `IConfigurationSection`, not a direct extension.

Fix: Use `GetValue<bool>(module)` consistently. This extension method returns the default value (false) if the key doesn't exist, avoiding exceptions for missing configuration.

Problem: `BusinessRuleValidationException` was not caught by the exception middleware because it wasn't in the `catch` chain.

Fix: Add it to the `ExceptionHandlerMiddleware.HandleExceptionAsync` switch before the default case. Without this, business rule violations resulted in HTTP 500 instead of HTTP 409.

Problem: The `CompletePurchaseEndpoint` injected `IEventBus` directly into the endpoint handler. This is fine for a simple module, but if you later decide to add event validation or compensation logic, you'd need to refactor.

Decision: Accept this trade-off. The Purchasing module's complexity doesn't justify a full Application layer.

Problem: Npgsql Entity Framework Core provider version must exactly match the EF Core version. Using Npgsql.EntityFrameworkCore.PostgreSQL 10.0.1 with EF Core 10.0.8 works because they follow the same major.minor version scheme.

Verification: Always check [NuGet](#) for the latest compatible version.

Problem: MediatR handlers were not being discovered because `AddMediatR` was called with the wrong assembly.

Fix: `services.AddMediatR(c => c.RegisterServicesFromAssembly(Assembly.GetExecutingAssembly()))` — the assembly must be the one containing the handlers, not the host project. In the Leasing module, handlers are in `LeaseBuyArch.Lessing.Application`, so `AddMediatR` is called in the Infrastructure project which references the Application project.

Problem: Dockerfile COPY commands reference paths relative to the solution root, but Docker build context was set to `Src/`. This caused "file not found" errors during Docker builds.

Fix: The `docker-compose.yml` used `build: .` where `.` is the `Src/` directory. All COPY paths in the Dockerfile must be relative to `Src/`. Alternatively, set build context to the repository root.

3.14 Additional Chapter 2 Discoveries

3.14.1 The Duplicate Module Record Bug

One of the most confusing issues was the **duplicate Module record**. The Ch2 codebase had TWO files:

- `Src/Common.Core/Module.cs` — defined `Module` record with module names in the shared kernel
- `Src/LeaseBuyArch/Module.cs` (host project) — defined the same `Module` record in the composition root

How it happened: The reference architecture evolved over time. Originally the `Module` record was in the shared kernel so any module could reference it. Later it was moved to the host project to avoid coupling. The old file was never deleted.

The bug: Both files define `Module` in slightly different namespaces (`EvolutionaryArchitecture.LeaseBuyArch.Common.Core` vs `EvolutionaryArchitecture.LeaseBuyArch`). When `Program.cs` uses `Module.Lessing`, the compiler can't resolve which `Module` class you meant, causing CS0246 or ambiguous reference errors.

The fix: Delete `Src/Common.Core/Module.cs`. The `Module` record belongs only in the host project.

3.14.2 The Two ModuleAvailabilityChecker Implementations

`ModuleAvailabilityChecker` existed in two places:

- `Src/Common.Infrastructure/` (standalone, outside Common folder)
- `Src/Common/LeaseBuyArch.Common.Infrastructure/` (inside Common folder)

They used different methods:

```
// Standalone version (WRONG):
return configuration.GetSection("FeatureManagement").Get<bool>(module.Value);

// Common version (CORRECT):
return configuration.GetSection("FeatureManagement").GetValue<bool>(module);
```

`Get<bool>()` is an extension on `IConfigurationSection`, not `IConfiguration`. Using it on `IConfiguration` directly compiles but fails at runtime. `GetValue<bool>()` is the correct method.

The fix: Remove the standalone `Common.Infrastructure/` version.

3.14.3 The `ILeasingModule` Abstraction

Chapter 2 introduces `ILeasingModule` to decouple the API layer from MediatR:

```
public interface ILeasingModule
{
    Task<TResult> ExecuteCommandAsync<TResult>(ICommand<TResult> command,
        CancellationToken ct = default);
    Task ExecuteCommandAsync(ICommand command, CancellationToken ct = default);
}

internal sealed class LeasingModule : ILeasingModule
{
    private readonly IMediator _mediator;

    public LeasingModule(IMediator mediator) => _mediator = mediator;

    public Task<TResult> ExecuteCommandAsync<TResult>(ICommand<TResult> command,
        CancellationToken ct) =>
        _mediator.Send(command, ct);

    public Task ExecuteCommandAsync(ICommand command, CancellationToken ct) =>
        _mediator.Send(command, ct);
}
```

Why this exists: The API layer (endpoints) should not depend on MediatR directly. `ILeasingModule` is an application-layer abstraction. Infrastructure wires up the MediatR implementation. This follows the Dependency Inversion Principle.

In the endpoint (no dependency on MediatR):

```
app.MapPost(LeasingApiPaths.Prepare, async (PrepareLeaseRequest request,
    ILeasingModule module, CancellationToken cancellationToken) =>
{
    var command = new PrepareLeaseCommand(...);
    var leaseId = await module.ExecuteCommandAsync(command, cancellationToken);
    return Results.Created($"{LeasingApiPaths.Prepare}/{leaseId}", leaseId);
});
```

3.14.4 The GlobalUsings.cs File

In addition to `Directory.Build.props` global usings, the host project has a `GlobalUsings.cs` :

```
global using System.Diagnostics.CodeAnalysis;
global using JetBrains.Annotations;
```

Why a separate file? `Directory.Build.props` global usings apply to ALL projects. `JetBrains.Annotations` is only needed in the host project (for `[UsedImplicitly]` on the `Program` class). Adding it to `Directory.Build.props` would force it on all 15 projects — unnecessary pollution.

4. Chapter 3 — Microservice Extraction: Splitting When Cohesion Demands It

4.1 Goal

Extract the **Leasing module** into its own standalone microservice. This is the most complex module — it has the richest domain model, the strictest business rules, and the highest change frequency. The remaining modules stay in the modular monolith but now communicate with Leasing via **RabbitMQ** instead of in-process MediatR.

4.2 Why Extract the Leasing Module?

The Leasing module is the best candidate for extraction because:

- 1. **High domain complexity** — Credit score validation, mileage limits, term limits, previous lease settlement checks, money factor calculations, residual value calculations, 14-day signing window. This module has the most business rules and is most likely to change independently.
- 2. **Different scalability requirements** — Lease signings might spike during promotional periods. A dedicated microservice can be scaled independently without scaling Purchasing or Vehicles.
- 3. **Clear bounded context** — The Leasing module has a well-defined boundary. It owns the Lease lifecycle from preparation to signing. Other modules only need to know about `LeaseSignedEvent`.
- 4. **Independent data** — The Leasing schema (`Leasing.Leases` table) is only written by the Leasing module. Other modules only read lease data indirectly through events.

4.3 What Changed

The transformation from Chapter 2 to Chapter 3 involved these specific changes:

Component	Chapter 2 (Modular Monolith)	Chapter 3 (Extracted)
Event bus	In-process MediatR	Out-of-process MassTransit/RabbitMQ
Leasing module	Project in monolith solution	Standalone solution, own host
Common layer	Project references	Project references (still shared)
Monolith Leasing ref	Direct project reference	Via <code>IntegrationEvents</code> project
Event handlers	<code>IIntegrationEventHandler<T></code>	<code>IConsumer<T></code> (MassTransit)

Module availability	IConfiguration	IFeatureManager
Host registration	AddLeasing(config, Module)	AddLeasing(config) (no feature gate)

4.4 Project Structure (25 projects across 3 solutions)

```

leasebuyarch3/
├── docker-compose.yml          # Postgres + RabbitMQ + both services
├── Makefile
├── LeaseBuyArch.Common/       # Solution 1: Shared packages (3 projects)
│   ├── Directory.Build.props
│   ├── LeaseBuyArch.Common.sln
│   ├── LeaseBuyArch.Common.Core/
│   ├── LeaseBuyArch.Common.Api/
│   └── LeaseBuyArch.Common.Infrastructure/ # ← MassTransit replaces MediatR here!
├── Leasing/                   # Solution 2: Extracted microservice (6 projects)
│   ├── Directory.Build.props
│   ├── LeaseBuyArch.Leasing.sln
│   ├── LeaseBuyArch.Leasing/   # New! Standalone host
│   │   ├── Program.cs
│   │   ├── appsettings.json
│   │   ├── appsettings.Development.json
│   │   └── Properties/launchSettings.json
│   ├── LeaseBuyArch.Leasing.Api/
│   ├── LeaseBuyArch.Leasing.Application/
│   ├── LeaseBuyArch.Leasing.Core/
│   ├── LeaseBuyArch.Leasing.Infrastructure/
│   └── LeaseBuyArch.Leasing.IntegrationEvents/ # Shared contract with monolith
├── ModularMonolith/          # Solution 3: The remaining monolith (11 projects)
│   ├── Directory.Build.props
│   ├── LeaseBuyArch.slnx
│   ├── Dockerfile
│   ├── LeaseBuyArch/          # Host (Leasing references removed)
│   │   ├── Program.cs
│   │   ├── Module.cs          # Only Purchasing, Vehicles, Comparison
│   │   ├── appsettings.json
│   │   ├── appsettings.Development.json
│   │   └── Properties/launchSettings.json
│   ├── Purchasing/            # Unchanged from Chapter 2
│   ├── Vehicles/              # Now has MassTransit consumers
│   │   ├── LeaseBuyArch.Vehicles.Api/
│   │   │   ├── VehiclesModule.cs
│   │   │   ├── VehiclesConsumers.cs
│   │   │   ├── ConsumeLeaseSignedEvent/
│   │   │   │   ├── LeaseSignedEventConsumer.cs
│   │   │   │   └── LeaseSignedEventRegistration.cs
│   │   │   └── ConsumePurchaseCompletedEvent/
│   │   │       └── PurchaseCompletedEventConsumer.cs

```

```

└─ PurchaseCompletedEventRegistration.cs
└─ LeaseBuyArch.Vehicles.DataAccess/
└─ Comparison/ # Unchanged from Chapter 2

```

4.5 The Event Bus Transformation (Most Important Change)

4.5.1 Old In-Memory Event Bus (Chapter 2)

```

// Infrastructure setup
public static IServiceCollection AddEventBus(this IServiceCollection services) =>
    services.AddInMemoryEventBus(Assembly.GetExecutingAssembly());

// In-memory implementation
public sealed class InMemoryEventBus : IEventBus
{
    private readonly IMediator _mediator;

    public async Task PublishAsync<TEvent>(TEvent @event, CancellationToken ct =
default)
        where TEvent : IIntegrationEvent =>
        await _mediator.Publish(@event, ct);
}

```

4.5.2 New MassTransit Event Bus (Chapter 3)

```

// Infrastructure setup (now takes configuration for RabbitMQ)
public static IServiceCollection AddCommonInfrastructure(
    this IServiceCollection services, IConfiguration configuration)
{
    services.AddEventBus(configuration);
    return services;
}

// MassTransit implementation
internal sealed class EventBus : IEventBus
{
    private readonly IPublishEndpoint _publishEndpoint;

    public EventBus(IPublishEndpoint publishEndpoint) => _publishEndpoint =
publishEndpoint;

    public async Task PublishAsync<TEvent>(TEvent @event, CancellationToken ct =
default)
        where TEvent : IIntegrationEvent =>
        await _publishEndpoint.Publish(@event, ct);
}

```

The critical insight: `IEventBus` interface does NOT change between Chapter 2 and Chapter 3. The interface is `PublishAsync<TEvent>(TEvent, CancellationToken)` in both versions. Only the

implementation changes. This is the **Dependency Inversion Principle** in action — abstractions (interfaces) are stable, implementations can vary.

4.5.3 MassTransit Configuration (The Complex Part)

```
internal static class EventBusModule
{
    internal static IServiceCollection AddEventBus(
        this IServiceCollection services, IConfiguration configuration)
    {
        // 1. Bind EventBusOptions from appsettings.json "EventBus" section
        services.Configure<EventBusOptions>(options =>
            configuration.GetSection("EventBus").Bind(options));

        // 2. Early-resolve registered ConsumerConfigurations
        //     These are registered by modules via RegisterConsumer()
        var serviceProvider = services.BuildServiceProvider();
        var endpoints = serviceProvider
            .GetRequiredService<IEnumerable<ConsumerConfiguration>>().ToList();

        // 3. Configure MassTransit with RabbitMQ
        services.AddMassTransit(configurator =>
        {
            RegisterConsumers(endpoints, configurator);

            configurator.UsingRabbitMq((context, factoryConfigurator) =>
            {
                var options = context.GetRequiredService<IOptions<EventBusOptions>>
                ();

                var eventBusOptions = options.Value;
                if (eventBusOptions.Uri is null ||
                    eventBusOptions.Username is null ||
                    eventBusOptions.Password is null)
                    return; // Can't connect without configuration

                factoryConfigurator.Host(eventBusOptions.Uri, h =>
                {
                    h.Username(eventBusOptions.Username);
                    h.Password(eventBusOptions.Password);
                });

                ConfigureConsumers(endpoints, factoryConfigurator, context);

                factoryConfigurator.ConfigureEndpoints(context);
            });
        });

        // 4. Register IEventBus as a scoped service wrapping MassTransit
        services.AddScoped<IEventBus, EventBus>();

        return services;
    }
}
```

```
}  
}
```

Step-by-step explanation:

1. **Configuration binding:** Reads `EventBus.Uri`, `EventBus.Username`, `EventBus.Password` from `appsettings.json`.
2. **Early service resolution:** This is the tricky part. `ConsumerConfiguration` instances are registered as singletons by modules calling `RegisterConsumer()`. But `AddEventBus` is called in `Program.cs` at service registration time — before the service provider is built. To get the registered `ConsumerConfiguration` instances, we need to call `BuildServiceProvider()` early, which is generally discouraged but acceptable here because:
 - `ConsumerConfiguration` is a simple data object with no dependencies
 - The early provider won't have all services registered yet, but we only need `IEnumerable<ConsumerConfiguration>`
 - This pattern is used in the reference architecture
3. **MassTransit configuration:** We register all consumers, configure RabbitMQ connection, configure receive endpoints (queues), and call `ConfigureEndpoints` to let MassTransit automatically bind exchanges to queues.
4. **EventBus registration:** `IEventBus` is registered as scoped so a new instance (with a new `IPublishEndpoint`) is created per HTTP request.

4.5.4 Consumer Registration Pattern

Each module registers its consumers:

```
// In Vehicles module API project:  
internal static class VehiclesConsumers  
{  
    internal static IServiceCollection AddConsumers(this IServiceCollection  
services)  
    {  
        services.RegisterLeaseSignedEventConsumer();  
        services.RegisterPurchaseCompletedEventConsumer();  
        return services;  
    }  
}  
  
// Registration for a specific event:  
internal static class LeaseSignedEventRegistration  
{  
    internal static IServiceCollection RegisterLeaseSignedEventConsumer(  
        this IServiceCollection services)  
    {  
        services.RegisterConsumer(  
            "leases-signed", // RabbitMQ queue name  
            typeof(LeaseSignedEventConsumer)); // MassTransit consumer type  
        return services;  
    }  
}
```

```

    }
}

```

The `RegisterConsumer` extension adds a `ConsumerConfiguration` singleton:

```

public static IServiceCollection RegisterConsumer(
    this IServiceCollection services, string queueName, Type consumerType)
{
    var consumerConfiguration = ConsumerConfiguration.Configure(queueName,
        consumerType);
    services.AddSingleton(consumerConfiguration);
    return services;
}

```

`ConsumerConfiguration` validates at construction that the type implements `IConsumer` :

```

internal sealed class ConsumerConfiguration
{
    private ConsumerConfiguration(string queueName, Type consumerType)
    {
        if (consumerType.GetInterface(nameof(IConsumer)) is null)
            throw new ArgumentException(
                $"{consumerType.FullName} must implement {typeof(IConsumer).FullName}",
                nameof(consumerType));

        QueueName = queueName;
        ConsumerType = consumerType;
    }

    internal static ConsumerConfiguration Configure(string queueName, Type
        consumerType) =>
        new(queueName, consumerType);

    internal string QueueName { get; }
    internal Type ConsumerType { get; }
}

```

The validation at construction time is important. It catches configuration errors (registering the wrong type as a consumer) at startup rather than at runtime when a message arrives.

4.6 The MassTransit Consumer

Instead of implementing `IIntegrationEventHandler<T>` , the consumer implements `IConsumer<T>` from MassTransit:

```

// Chapter 2 (in-process):
internal sealed class LeaseSignedEventHandler :
    IIntegrationEventHandler<LeaseSignedEvent>
{

```

```

    public async Task Handle(LeaseSignedEvent @event, CancellationToken
cancellationTokentoken)
    {
        // Process the event
    }
}

// Chapter 3 (out-of-process via RabbitMQ):
internal sealed class LeaseSignedEventConsumer : IConsumer<LeaseSignedEvent>
{
    private readonly VehiclesPersistence _persistence;

    public LeaseSignedEventConsumer(VehiclesPersistence persistence) => _persistence
= persistence;

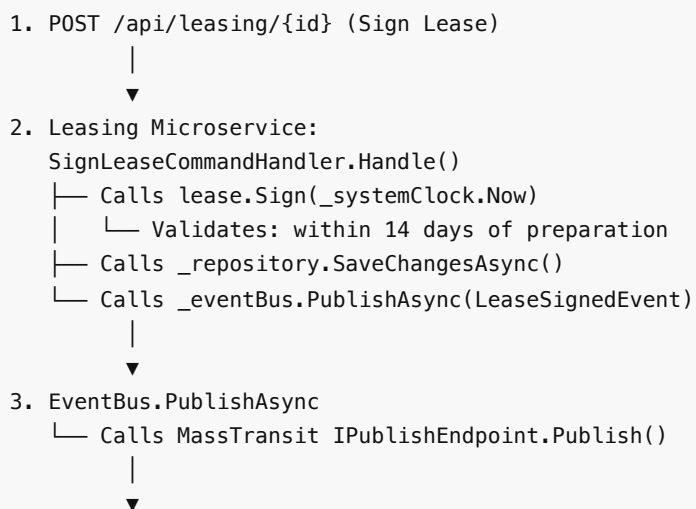
    public async Task Consume(ConsumeContext<LeaseSignedEvent> context)
    {
        var @event = context.Message;
        var vehicle = await _persistence.Vehicles.FindAsync(
            new object?[] { @event.VehicleId }, context.CancellationToken);
        if (vehicle is null) return;
        vehicle.MarkAsLeased();
        await _persistence.SaveChangesAsync(context.CancellationToken);
    }
}

```

Key differences from the Chapter 2 handler:

1. `Consume` receives `ConsumeContext<T>` instead of raw `T`. The context provides message metadata, cancellation, and the ability to retry/reject.
2. The event is accessed via `context.Message`.
3. The consumer is discovered and registered by MassTransit, not MediatR.
4. If the consumer throws an exception, MassTransit will retry the message (configurable via retry policies).

4.7 The Complete End-to-End Event Flow



4. RabbitMQ:
- └─ Receives message on default exchange
 - └─ Routes to "leases-signed" queue (bound by MassTransit)
 - └─ Message persists until consumed
- ▼
5. ModularMonolith:
- LeaseSignedEventConsumer.Consume()
 - └─ Loads vehicle from Vehicles DB
 - └─ Calls vehicle.MarkAsLeased()
 - └─ Calls _persistence.SaveChangesAsync()

4.8 The Leasing Microservice Host

The Leasing microservice has its own `Program.cs` :

```
using EvolutionaryArchitecture.LeaseBuyArch.Common.Api.ErrorHandling;
using EvolutionaryArchitecture.LeaseBuyArch.Common.Core.SystemClock;
using EvolutionaryArchitecture.LeaseBuyArch.Common.Infrastructure;
using EvolutionaryArchitecture.LeaseBuyArch.Leasing.Api;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddSystemClock();
builder.Services.AddLeasing(builder.Configuration);
builder.Services.AddCommonInfrastructure(builder.Configuration); // ← MassTransit
setup

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.UseErrorHandling();
app.MapControllers();
app.RegisterLeasing();

app.Run();
```

Notable absences compared to the monolith's Program.cs:

- No `AddFeatureManagement()` — the microservice only has one module (Leasing), always enabled
- No `Module` record — not needed in a single-module service

- No `IsModuleEnabled` checks — the Leasing module is the entire service
- No `AddPurchasing` , `AddVehicles` , `AddComparison` — those stay in the monolith

However, `AddCommonInfrastructure` **IS still called**. The microservice needs MassTransit to **publish** events to RabbitMQ, even though it doesn't consume any events from other services.

4.9 The Monolith's Module Record (Leasing Removed)

```
internal record Module(string Value)
{
    internal static readonly Module Purchasing = new("Purchasing");
    internal static readonly Module Vehicles = new("Vehicles");
    internal static readonly Module Comparison = new("Comparison");
    // NOTE: Leasing is no longer here. It's now a separate service.
}
```

4.10 The Modular Monolith Host Program.cs (Leasing Removed)

```
using EvolutionaryArchitecture.LeaseBuyArch;
using EvolutionaryArchitecture.LeaseBuyArch.Common.Api.ErrorHandling;
using EvolutionaryArchitecture.LeaseBuyArch.Common.Core.SystemClock;
using EvolutionaryArchitecture.LeaseBuyArch.Common.Infrastructure;
using EvolutionaryArchitecture.LeaseBuyArch.Purchasing.Api;
using EvolutionaryArchitecture.LeaseBuyArch.Vehicles.Api;
using EvolutionaryArchitecture.LeaseBuyArch.Comparison;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddSystemClock();
builder.Services.AddFeatureManagement();           // ← MUST be before
AddCommonInfrastructure!

builder.Services.AddPurchasing(config, Module.Purchasing);
builder.Services.AddVehicles(config, Module.Vehicles);           // ← registers
MassTransit consumers
builder.Services.AddComparison(config, Module.Comparison);
builder.Services.AddCommonInfrastructure(config);                 // ← MassTransit setup
(reads consumers)

var app = builder.Build();

app.UseErrorHandling();
app.MapControllers();

app.RegisterPurchasing(Module.Purchasing);
app.RegisterVehicles(Module.Vehicles);
```

```
app.RegisterComparison(Module.Comparison);

app.Run();
```

4.11 The appsettings.json (Now with EventBus Section)

Both the monolith and the Leasing microservice have the `EventBus` configuration section:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "Purchasing": "",
    "Vehicles": ""
  },
  "EventBus": {
    "Uri": "localhost",
    "Username": "guest",
    "Password": "guest"
  }
}
```

Development override (changes the URI to the Docker container name):

```
{
  "EventBus": {
    "Uri": "rabbitmq",
    "Username": "guest",
    "Password": "guest"
  },
  "ConnectionStrings": {
    "Purchasing":
      "Host=postgres:5432;Database=leasebuy;Username=postgres;Password=mysecretpassword",
    "Vehicles":
      "Host=postgres:5432;Database=leasebuy;Username=postgres;Password=mysecretpassword"
  }
}
```

Why `rabbitmq` as the URI? In Docker Compose, services communicate by their service name. The RabbitMQ service is named `rabbitmq`, so `rabbitmq:5672` is the connection string. When running locally (without Docker), it would be `localhost:5672`.

4.12 The docker-compose.yml (Now with RabbitMQ)

```

version: '3.9'

services:
  leasebuyarch-modular-monolith:
    build:
      context: . # Root context so Dockerfiles can access Common/
      dockerfile: ModularMonolith/Dockerfile
    ports:
      - "8080:80"
    depends_on:
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy # ← New dependency

  leasebuyarch-leasing-microservice:
    build:
      context: .
      dockerfile: Leasing/Dockerfile
    ports:
      - "8081:80" # ← Different port from monolith
    depends_on:
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy

  postgres:
    image: postgres:14.3
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_PASSWORD=mysecretpassword
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]

  rabbitmq:
    image: rabbitmq:management
    ports:
      - "15672:15672" # Management UI
      - "5672:5672" # AMQP protocol
    healthcheck:
      test: ["CMD", "rabbitmqctl", "ping"]

```

Key changes from Chapter 2's docker-compose:

1. **Two application services** instead of one — monolith (8080) and Leasing (8081)
2. **RabbitMQ service** added with management UI (15672) and AMQP (5672)
3. **Build context is** `.` (root of leasebuyarch3/) instead of subdirectory, so Dockerfiles can access Common projects
4. **Both services depend on RabbitMQ** with health check (not just startup)

4.13 Consumer Queue Names and RabbitMQ Topology

MassTransit automatically creates:

1. An **exchange** for each event type (e.g., `LeaseSignedEvent` exchange)
2. A **queue** for each consumer registration (e.g., `leases-signed` queue)
3. A **binding** from exchange to queue

Queue names used:

Queue Name	Event Type	Consumer	Service
leases-signed	LeaseSignedEvent	LeaseSignedEventConsumer	Monolith (Vehicles)
purchases-completed	PurchaseCompletedEvent	PurchaseCompletedEventConsumer	Monolith (Vehicles)

Why two separate queues? Each queue represents a different message stream. If both consumers used the same queue, they would compete for messages (competing consumers pattern). That's useful for scaling, but here each consumer handles a different event type.

4.14 Module Availability Checker: IConfiguration → IFeatureManager

In Chapter 2, we used `IConfiguration` directly:

```
public static bool IsModuleEnabled(this IConfiguration configuration, string module)
{
    var featureManagement = configuration.GetSection("FeatureManagement");
    return featureManagement.GetValue<bool>(module);
}
```

In Chapter 3, the reference uses `IFeatureManager` from `Microsoft.FeatureManagement` :

```
public static bool IsModuleEnabled(this IServiceCollection services, string module)
{
    var serviceProvider = services.BuildServiceProvider();
    var featureManager = serviceProvider.GetRequiredService<IFeatureManager>();
    return featureManager.IsEnabledAsync(module).Result; // .Result = code smell
}

public static bool IsModuleEnabled(this IApplicationBuilder app, string module)
{
    var serviceProvider = app.ApplicationServices;
    var featureManager = serviceProvider.GetRequiredService<IFeatureManager>();
    return featureManager.IsEnabledAsync(module).Result;
}
```

The `.Result` call is a code smell. It blocks the calling thread to synchronously get an async result. This can cause deadlocks in ASP.NET Core synchronization contexts. However, it's acceptable here because:

- It's called during service registration, which is inherently synchronous

- The `IsEnabledAsync` implementation is typically fast (in-memory cache lookup)
- The risk of deadlock is low because there's no `ConfigureAwait(false)` issue at service registration time

Why the change? `IFeatureManager` supports more sophisticated feature flag configurations:

- Percentage-based rollouts (enable for 10% of users)
- Time window filters (enable only during business hours)
- Custom filters (enable for users in a specific region)
- Dynamic updates (change flags without restarting)

For simple on/off toggles, `IConfiguration` is sufficient. The `IFeatureManager` approach is more future-proof.

4.15 Technology Fixes in Chapter 3

Problem: NU1510 error: "PackageReference will not be pruned."

Cause: In .NET 10, the ASP.NET Core shared framework automatically includes

`Microsoft.Extensions.DependencyInjection.Abstractions` ,
`Microsoft.AspNetCore.Http.Abstractions` ,
`Microsoft.Extensions.Configuration.Abstractions` , and
`Microsoft.Extensions.Options.ConfigurationExtensions` . Adding them explicitly as NuGet

references causes prune warnings, which become errors due to

`<TreatWarningsAsErrors>true</TreatWarningsAsErrors>` .

Fix: Remove explicit package references for anything provided by the ASP.NET Core shared framework. The specific packages to remove were:

- `Microsoft.Extensions.DependencyInjection.Abstractions`
- `Microsoft.AspNetCore.Http.Abstractions`
- `Microsoft.Extensions.Configuration.Abstractions`
- `Microsoft.Extensions.Options.ConfigurationExtensions`

Problem: CS8604: Possible null reference argument in EventBusModule.

Cause: `EventBusOptions.Uri` , `.Username` , `.Password` are declared as `string?` (nullable) but `IRabbitMqHostConfigurator.Username()` and `.Password()` accept `string` (non-nullable).

Fix: Add a null guard: `if (eventBusOptions.Uri is null || eventBusOptions.Username is null || eventBusOptions.Password is null) return;`

Problem: The Dockerfile COPY commands broke because of build context mismatch.

Cause: The original docker-compose set `build: ./ModularMonolith` which made the Dockerfile's context the `ModularMonolith/` directory. But the Dockerfile referenced `../LeaseBuyArch.Common/...` paths that are outside this context. Docker's COPY command cannot access files outside the build context.

Fix: Set the build context to the repository root (`context: .`) and specify the Dockerfile path explicitly (`dockerfile: ModularMonolith/Dockerfile`). Then all COPY paths are relative to the root, and `Common/` files are accessible.

Problem: `IFeatureManager` not available when `AddCommonInfrastructure` is called.

Cause: `AddCommonInfrastructure` calls

`services.BuildServiceProvider().GetRequiredService<IEnumerable<ConsumerConfiguration>>()` . If `AddFeatureManagement()` hasn't been called yet, the service provider builder hasn't registered `IFeatureManager` , and the early `BuildServiceProvider()` may throw or return incorrect results.

Fix: Ensure `AddFeatureManagement()` is called **before** `AddCommonInfrastructure()` in `Program.cs` . The order matters:

```
// CORRECT:
services.AddFeatureManagement();
services.AddCommonInfrastructure(configuration);

// WRONG (will fail or behave incorrectly):
services.AddCommonInfrastructure(configuration);
services.AddFeatureManagement();
```

Problem: Cross-solution project references in .slnx files.

Cause: The monolith needs to reference `LeaseBuyArch.Leasing.IntegrationEvents` (in the Leasing solution) for the `LeaseSignedEvent` type used by its consumers. But .slnx files are solution-scoped — they don't know about projects in other solutions.

Fix: Direct project references work across solution boundaries. The `.csproj` file can reference any project on disk:

```
<!-- In ModularMonolith/Vehicles/LeaseBuyArch.Vehicles.Api.csproj -->
<ProjectReference
  Include="..\..\..\Leasing\LeaseBuyArch.Leasing.IntegrationEvents\LeaseBuyArch.Leasing..
/>
```

MSBuild resolves this reference regardless of solution boundaries. The `.slnx` file only affects the Visual Studio/Rider IDE experience — it defines which projects appear in the solution explorer — but it doesn't restrict which projects can reference each other.

Problem: `IIntegrationEvent` extends MediatR's `INotification`, but MassTransit events don't need it. In Chapter 3, the `InMemoryEventBus` is gone, so `INotification` on events is vestigial.

Fix: Keep `INotification` on `IIntegrationEvent`. It's harmless because no code uses MediatR dispatch for these events anymore. Removing it would break the `IIntegrationEventHandler` interface chain. The pattern is kept for backward compatibility with any handlers that might still use MediatR directly.

Problem: The `AddEventBus` extension uses `services.BuildServiceProvider()` during service registration, which is generally discouraged.

Risk: The early service provider only has the services registered up to that point. If `ConsumerConfiguration` is registered after `AddCommonInfrastructure`, it won't be found.

Mitigation: The module's `AddVehicles` (which calls `AddConsumers` → `RegisterConsumer`) must be called **before** `AddCommonInfrastructure`. This is an ordering constraint that must be maintained.

Problem: Multiple `Directory.Build.props` files across the three solutions.

Cause: Each solution has different projects and needs different global usings. The Common solution's `Directory.Build.props` includes ASP.NET global usings, but `Common.Core` doesn't have the ASP.NET framework reference.

Fix: Add `<FrameworkReference Include="Microsoft.AspNetCore.App" />` to each project that needs ASP.NET types (Core, Api, Infrastructure). The shared `Directory.Build.props` global usings are harmless for projects that don't use them — they're just unused imports.

4.16 Additional Chapter 3 Discoveries

4.16.1 The EventBusModule Null Guard Evolution

The initial `EventBusModule.AddEventBus` had a null safety problem:

```
// What we originally wrote (CS8604 warning):
configurator.UsingRabbitMq((context, factoryConfigurator) =>
{
    var options = context.GetRequiredService<IOptions<EventBusOptions>>().Value;
    factoryConfigurator.Host(options.Uri, h =>
    {
        h.Username(options.Username);
        h.Password(options.Password);
    });
});
```

Problem: `EventBusOptions.Uri`, `.Username`, `.Password` are `string?` (nullable for unset config), but `IRabbitMqHostConfigurator` methods accept `string` (non-nullable). This is CS8604: "Possible null reference argument."

Fix evolution:

```
// Fix 1 (null-forgiving operator – works but dangerous):
factoryConfigurator.Host(options.Uri!, h => { h.Username(options.Username!); ... });

// Fix 2 (null guard – correct):
if (options.Value.Uri is null || options.Value.Username is null ||
options.Value.Password is null) return;
```

Why Fix 2 is better: The null-forgiving operator just tells the compiler "trust me." If it IS null at runtime, you get a `NullReferenceException` with no context. The null guard provides early, explicit failure.

4.16.2 The `ResourceNotFoundException`

Chapter 3's `Common.Api` adds `ResourceNotFoundException` that doesn't exist in earlier chapters:

```
public sealed class ResourceNotFoundException : InvalidOperationException
{
    public ResourceNotFoundException(Guid id) : base($"Resource with '{id}' not
found ") { }
}
```

The `ExceptionHandlerMiddleware` catches it and returns HTTP 404:

```
case ResourceNotFoundException resourceNotFoundException:
    statusCode = (int)HttpStatusCode.NotFound;
    message = resourceNotFoundException.Message;
    break;
```

Why this was added: In Chapter 2, when a lease wasn't found, the handler threw `InvalidOperationException` which resulted in HTTP 500. But "not found" is a client error (4xx), not a server error (5xx). This provides proper 404 responses.

4.16.3 The `Common Version Property`

Chapter 3's `Common` `Directory.Build.props` includes a version:

```
<PropertyGroup>
  <Version>1.1.5</Version>
</PropertyGroup>
```

This is a signal that Common is intended to be published as a NuGet package — versioning metadata is only needed for packages. Neither Leasing nor ModularMonolith `Directory.Build.props` includes a `<Version>` property.

Why this matters: When you eventually publish Common as NuGet packages (per ADR-1), the version is already configured. Just run `dotnet pack` to produce `.nupkg` files.

4.16.4 Mixed .sln and .slnx Formats

The Ch2 ModularMonolith uses `.slnx` (new .NET 10 format), but the Ch3 Common solution still uses the old `.sln` format:

```
Microsoft Visual Studio Solution File, Format Version 12.00
Project("{FAE04EC0-301F-11D3-BF4B-00C04F79EFBC}") = "LeaseBuyArch.Common.Core", ...
```

Why the inconsistency: `.slnx` became the default in .NET 10 SDK. The Common solution was created before we standardized on `.slnx`, or the template default changed during development. Both formats work identically for building — `.slnx` just provides better merge resolution and no GUIDs.

4.16.5 The Module Registration Pattern (No Feature Gate)

In Chapter 3, the Leasing module registration no longer has a feature flag:

```
// Ch2 (with feature gate):
public static IServiceCollection AddLeasing(this IServiceCollection services,
    IConfiguration configuration, string module)
{
    if (!configuration.IsModuleEnabled(module)) return services;
    services.AddLeasingInfrastructure(configuration);
    return services;
}

// Ch3 (no feature gate – Leasing is its own service):
public static IServiceCollection AddLeasing(this IServiceCollection services,
    IConfiguration configuration)
{
    services.AddLeasingInfrastructure(configuration);
    return services;
}
```

Why the difference: In Chapter 2, Leasing is one module among many in a monolith — you might want to disable it. In Chapter 3, Leasing IS the entire service. Disabling it would mean the service does nothing. Feature flags on a per-service basis are handled at the deployment level (start/stop the container), not in code.

5. The Leasing Calculation Engine

5.1 How Leasing Math Works

A car lease payment consists of two components:

1. **Depreciation Fee:** The portion of the vehicle's value that you use during the lease term.

- Formula: $(\text{MSRP} - \text{Residual Value}) / \text{Term Months}$
- Where $\text{Residual Value} = \text{MSRP} \times (\text{Residual Percentage} / 100)$

2. **Rent Charge:** The financing cost (like interest on a loan).

- Formula: $(\text{MSRP} + \text{Residual Value}) \times \text{Money Factor}$
- Money Factor is a decimal (e.g., 0.00125) that represents the interest rate

3. **Monthly Payment:** $\text{Depreciation Fee} + \text{Rent Charge}$

```
private static decimal CalculateMonthlyPayment(decimal msrp, decimal
residualPercentage,
    decimal moneyFactor, int termMonths)
{
    var residualValue = msrp * (residualPercentage / 100);
    var depreciation = msrp - residualValue;
    var depreciationFee = depreciation / termMonths;
    var rentCharge = (msrp + residualValue) * moneyFactor;
    return Math.Round(depreciationFee + rentCharge, 2);
}
```

5.2 How Purchasing Math Works

A car loan payment (purchase financing) uses the standard amortization formula:

```
private static decimal CalculateMonthlyPayment(decimal msrp, decimal downPayment,
    decimal apr, int termMonths)
{
    var loanAmount = msrp - downPayment;
    var monthlyRate = apr / 100 / 12;
    if (monthlyRate == 0)
        return Math.Round(loanAmount / termMonths, 2);
    var factor = (decimal)Math.Pow(1 + (double)monthlyRate, termMonths);
    var payment = loanAmount * (monthlyRate * factor) / (factor - 1);
    return Math.Round(payment, 2);
}
```

5.3 Business Rules for Leasing

The `Lease.Prepare()` factory method validates four business rules:

```
public static Lease Prepare(..., int creditScore, ..., int annualMileageLimit,
    int termMonths, ..., bool? isPreviousLeaseSettled = null)
{
    BusinessRuleValidator.Validate(new CustomerCreditScoreMustBeHighEnoughRule(
        creditScore, MinCreditScore));    // Credit score >= 700
}
```

```

        BusinessRuleValidator.Validate(new MileageMustNotExceedMaxLimitRule(
            annualMileageLimit, MaxAnnualMileage)); // Mileage <= 15,000
        BusinessRuleValidator.Validate(new LeaseTermMustNotExceedMaxRule(
            termMonths, MaxTermMonths));           // Term <= 36 months
        BusinessRuleValidator.Validate(new PreviousLeaseMustBeSettledRule(
            isPreviousLeaseSettled));               // Previous lease must be paid

        var monthlyPayment = CalculateMonthlyPayment(...);
        return new Lease(..., monthlyPayment, ...);
    }

```

And the `Sign()` method validates one more rule:

```

public void Sign(DateTimeOffset signedAt)
{
    BusinessRuleValidator.Validate(
        new LeaseCanOnlyBeSignedWithin14DaysFromPreparation(PreparedAt, signedAt));
    SignedAt = signedAt;
}

```

5.4 Concrete Calculation Examples

Example 1: Leasing a \$45,000 vehicle

Parameters:

- MSRP: \$45,000
- Residual percentage: 55% (residual value = \$24,750 after 36 months)
- Money factor: 0.00125 (equivalent to 3% APR)
- Term: 36 months

Step-by-step:

Residual Value	= \$45,000 × 0.55	= \$24,750.00
Depreciation	= \$45,000 - \$24,750	= \$20,250.00
Depreciation Fee	= \$20,250 / 36	= \$562.50
Rent Charge	= (\$45,000 + \$24,750) × 0.00125	= \$87.19
Monthly Payment	= \$562.50 + \$87.19	= \$649.69

Example 2: Purchasing the same \$45,000 vehicle

Parameters:

- MSRP: \$45,000
- Down payment: \$5,000
- APR: 4.5%
- Term: 60 months

Step-by-step:

Loan Amount	= \$45,000 - \$5,000	= \$40,000.00
Monthly Rate	= 4.5% / 12	= 0.00375
Factor	= (1 + 0.00375)^60	≈ 1.2516

$$\begin{aligned} \text{Payment} &= \$40,000 \times (0.00375 \times 1.2516) / (1.2516 - 1) \\ \text{Monthly Payment} &\approx \$745.68 \end{aligned}$$

Comparison: Leasing costs **\$649.69/month** for 36 months (total: \$23,388.84), while purchasing costs **\$745.68/month** for 60 months (total: \$44,740.80). The lease has lower payments but no equity. The purchase builds equity but costs more per month.

5.5 The Money Factor Explained

Money factor is the decimal representation of the interest rate on a lease:

$$\begin{aligned} \text{APR} &= \text{Money Factor} \times 2,400 \\ \text{Money Factor} &= \text{APR} / 2,400 \end{aligned}$$

APR	Money Factor
3.0%	0.00125
4.8%	0.00200
7.2%	0.00300
9.6%	0.00400

Why 2,400? The formula derives from the lease calculation: $(\text{MSRP} + \text{Residual}) \times \text{Money Factor}$. The 2,400 factor converts annual percentage rate to a monthly decimal compatible with this formula (12 months $\times$ 200 basis points per percent = 2,400).

5.6 Edge Cases in the Calculation

Zero money factor: If a promotion offers 0% APR, the money factor is 0. The rent charge becomes \$0. Monthly payment is just $\text{depreciation} / \text{termMonths}$.

Zero down payment (purchasing): When both $\text{downPayment} = 0$ and $\text{apr} = 0$, the formula returns $\text{loanAmount} / \text{termMonths}$ to avoid division by zero (the amortization denominator $\text{factor} - 1 = 0$ when $\text{monthlyRate} = 0$).

Floating-point precision: `Math.Pow` uses `double` arithmetic, introducing ~\$0.01-\$0.05 imprecision for typical auto loans. Acceptable for consumer-facing estimates, but not for financial accounting systems.

5.7 Complete Leasing Business Rules

The `Lease.Prepare()` factory validates four rules before creating a lease:

Rule	Constraint	Error Message
<code>CustomerCreditScoreMustBeHighEnoughRule</code>	Credit score ≥ 700	"Customer credit score {score} is below the minimum requirement of 700"
<code>MileageMustNotExceedMaxLimitRule</code>	Annual mileage $\leq 15,000$	"Annual mileage limit {limit} exceeds maximum allowed 15000"

LeaseTermMustNotExceedMaxRule	Term ≤ 36 months	"Lease term {term} months exceeds maximum 36 months"
PreviousLeaseMustBeSettledRule	Previous lease paid or no previous lease	"Previous lease must be settled by the customer"

The `Sign()` method adds a fifth rule:

Rule	Constraint	Error Message
LeaseCanOnlyBeSignedWithin14DaysFromPreparation	Signed within 14 days of preparation	"Lease can not be signed because more than 14 days have passed from the lease preparation"

Each rule is a separate class implementing `IBusinessRule`, enabling independent unit testing and composition.

6. Technology Deep Dive — Every Fix, Every Surprise

6.1 .NET 10 SDK and Runtime

Version discovered: `10.0.300` Location: `/usr/local/share/dotnet`

Surprises:

- `.slnx` format is the default when creating new solutions with `dotnet new sln`. The old `.sln` format is deprecated but still supported.
- `dotnet workload list` shows available workloads. For ASP.NET Core development, no additional workload is needed beyond the base SDK.
- The `Microsoft.NET.Sdk.Web` SDK is required for web projects. Class libraries use `Microsoft.NET.Sdk`.
- The SDK path was not in the default `$PATH` on macOS. Always use the full path or export it: `export PATH="/usr/local/share/dotnet:$PATH"`.

Tip: Add the export to your `.zshrc` or `.bashrc` so you don't have to type it every time.

6.2 NuGet Package Version Matrix

The exact versions that worked together:

Package	Version	Notes
Microsoft.NET.Sdk (built-in)	10.0.300	Targets .NET 10
MassTransit	8.3.0	Must match MassTransit.RabbitMQ version
MassTransit.RabbitMQ	8.3.0	Must match MassTransit version
MediatR	12.0.1	Stable version for .NET 10
Microsoft.EntityFrameworkCore	10.0.8	Preview version for .NET 10
Npgsql.EntityFrameworkCore.PostgreSQL	10.0.1	Preview, matches EF Core 10.x

Microsoft.FeatureManagement	4.0.0	Latest stable
Swashbuckle.AspNetCore	6.4.0	Swagger UI and OpenAPI
Microsoft.AspNetCore.OpenApi	10.0.8	.NET 10 OpenAPI support
Dapper	2.0.123	Micro-ORM, no version conflicts
Npgsql	9.0.3	ADO.NET PostgreSQL driver
JetBrains.Annotations	2022.3.1	[UsedImplicitly] attribute

6.3 The TreatWarningsAsErrors Saga

This setting in `Directory.Build.props` caused more build failures than actual code errors. Here is every warning type that bit us:

Warning ID	Cause	How We Fixed It
NU1510	Package pruning — a NuGet package is already included by the framework	Remove the explicit <code><PackageReference></code> from <code>.csproj</code>
NU1504	Duplicate <code>PackageReference</code> in <code>.csproj</code>	Check for duplicate lines and remove one
CS8604	Possible null reference passed to method expecting non-nullable	Add null guard (<code>if (x is null) return;</code>) or use null-forgiving operator (<code>x!</code>)
CS0246	Type or namespace not found	Add <code>using</code> directive, add <code>FrameworkReference</code> , or add missing NuGet package
CS1061	Extension method not found (e.g., <code>AddFeatureManagement</code>)	Add <code>using Microsoft.FeatureManagement;</code> or add NuGet package reference

6.4 Global Usings: Blessing and Curse

The `Directory.Build.props` global usings:

```
<Using Include="Microsoft.AspNetCore.Builder" />
<Using Include="Microsoft.AspNetCore.Http" />
<Using Include="Microsoft.AspNetCore.Routing" />
<Using Include="Microsoft.Extensions.Configuration" />
<Using Include="Microsoft.Extensions.DependencyInjection" />
```

Why this is great: Every `.cs` file in every project has `IApplicationBuilder`, `HttpContext`, `IConfiguration`, `IServiceCollection`, `WebApplication` available without writing `using` statements. This eliminates hundreds of lines of boilerplate across 25 projects.

Why this is dangerous: When you add `<FrameworkReference Include="Microsoft.AspNetCore.App" />` to a project like `Common.Core`, you force it to depend on the entire ASP.NET Core shared framework. This is a **leaky abstraction** — a core domain library should ideally be pure C# without web dependencies.

Our approach: Accept the leaky abstraction for now. The reference architecture avoids this by publishing `Common` as NuGet packages — the NuGet packages can have different dependency chains than the

consuming projects. If we were to publish Common as NuGet packages, we would only include the necessary dependencies.

6.5 MassTransit Version Compatibility

MassTransit 8.3.0 was the latest version compatible with .NET 10 at the time of writing. Key configuration points:

- `AddMassTransit()` registers MassTransit services in the DI container
- `UsingRabbitMq()` configures the RabbitMQ transport
- `ConfigureEndpoints()` automatically creates exchanges, queues, and bindings
- `IPublishEndpoint` publishes messages to exchanges (fan-out by default)
- `IConsumer<T>` is the consumer interface — MassTransit instantiates consumers via DI

RabbitMQ topology created by MassTransit:

1. Exchange: `LeaseSignedEvent` (type: fan-out, named after the message type's full name)
2. Queue: `leases-signed` (named in our `RegisterConsumer` call)
3. Binding: exchanges. `LeaseSignedEvent` exchange → `leases-signed` queue

MassTransit naming conventions:

- Exchange names use the full type name:
`EvolutionaryArchitecture.LeaseBuyArch.Leasing.IntegrationEvents.LeaseSignedEvent`
- Queue names are whatever we specify in `RegisterConsumer`
- Binding keys match the exchange name

6.6 RabbitMQ Management UI

When RabbitMQ is running (via Docker Compose), the management UI is available at:

- **URL:** <http://localhost:15672>
- **Username:** guest
- **Password:** guest

Use the management UI to:

- View queues and their message counts
- Inspect messages (payload, headers, properties)
- Bind/unbind exchanges and queues
- Monitor connection status
- Force message delivery (useful for debugging)

6.7 PostgreSQL Schema Design

Each module's schema is defined in its `DbContext.OnModelCreating` :

Module	Schema	Table	Connection String
Leasing	Leasing	Leases	ConnectionStrings:Leasing
Purchasing	Purchasing	Purchases	ConnectionStrings:Purchasing
Vehicles	Vehicles	Vehicles	ConnectionStrings:Vehicles

The Comparison module does NOT use EF Core — it uses Dapper for direct SQL queries, so it doesn't have a schema.

Why schemas instead of separate databases? Running multiple PostgreSQL databases would require:

- Multiple connection strings (more configuration)
- Multiple database instances (more resource usage)
- Complex cross-database queries (requires foreign data wrappers or application-level joins)

Schemas provide logical separation with a single connection string and a single database instance. When the Leasing module was extracted, it kept its own database configuration but pointed to the same PostgreSQL instance. In production, it could easily point to a different instance.

6.8 Migration Strategy

Each EF Core DbContext generates its own migrations. To add a migration:

```
# From the solution root:
dotnet ef migrations add CreateLeasesTable \
  --context LeasingPersistence \
  --project Leasing/LeaseBuyArch.Leasing.Infrastructure \
  --startup-project Leasing/LeaseBuyArch.Leasing

dotnet ef migrations add CreatePurchasesTable \
  --context PurchasingPersistence \
  --project ModularMonolith/Purchasing/LeaseBuyArch.Purchasing.DataAccess \
  --startup-project ModularMonolith/LeaseBuyArch
```

Important: You must specify `--startup-project` because EF Core tools need to build the project to discover the DbContext. The startup project must reference the project containing the DbContext.

At startup, migrations are applied automatically:

```
public static WebApplication UseDatabase(this WebApplication app)
{
    using var scope = app.Services.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<VehiclesPersistence>();
    context.Database.Migrate();
    return app;
}
```

Risk of auto-migration: In production, auto-migrating at startup is dangerous because:

- If the migration fails, the application fails to start
- If you have multiple instances, they all try to migrate simultaneously
- You can't review the migration before it runs

Production recommendation: Use `dotnet ef migrations script` to generate SQL scripts, review them in CI/CD, and apply them as a separate deployment step.

6.9 The Makefile

The `Makefile` at the root of `leasebuyarch3/` orchestrates all three solutions:

```
.PHONY: all help restore build build-common build-leasing build-monolith \
        test run run-monolith run-leasing clean \
```

```
docker-build docker-up docker-down docker-logs
```

```
DOTNET := dotnet
COMMON_DIR := LeaseBuyArch.Common
COMMON_SLN := $(COMMON_DIR)/LeaseBuyArch.Common.sln
LEASING_DIR := Leasing
LEASING_SLN := $(LEASING_DIR)/LeaseBuyArch.Leasing.sln
MONOLITH_DIR := ModularMonolith
MONOLITH_SLN := $(MONOLITH_DIR)/LeaseBuyArch.slnx
```

all: restore build

restore:

```
$(DOTNET) restore $(COMMON_SLN)
$(DOTNET) restore $(LEASING_SLN)
$(DOTNET) restore $(MONOLITH_SLN)
```

build: restore

```
$(DOTNET) build $(COMMON_SLN) --no-restore
$(DOTNET) build $(LEASING_SLN) --no-restore
$(DOTNET) build $(MONOLITH_SLN) --no-restore
```

Why `--no-restore` ? The `restore` target runs first and restores all three solutions. Without `--no-restore`, each `dotnet build` would run `restore` again, which is redundant and slow.

Build order matters: Common must build first (everything depends on it). Leasing and Monolith can build in parallel (they depend on Common but not on each other). Since Make runs commands sequentially, we list Common first.

Why three separate `dotnet build` commands instead of a single solution? Because the three solutions are independent — they don't share a `.slnx` file. Each solution has its own `Directory.Build.props` and its own set of projects.

6.10 .NET 10 OpenAPI Changes

In .NET 10, the `WithOpenApi()` extension method was replaced with `AddOpenApiOperationTransformer` :

```
// Old (.NET 7-9):
app.MapPost("/api/leasing", ...)
    .WithOpenApi(operation =>
    {
        operation.Summary = "Prepares a new lease";
        return operation;
    });

// New (.NET 10):
app.MapPost("/api/leasing", ...)
    .AddOpenApiOperationTransformer((operation, context, ct) =>
    {
        operation.Summary = "Prepares a new lease";
```

```
        return Task.CompletedTask;
    });
```

Why the change: The new pattern separates operation transformers from endpoint definitions. Multiple transformers can be applied to the same endpoint. Transformers can be reused across endpoints. The old pattern required modifying the operation inline.

Return type difference: The old pattern returned `OpenApiOperation`. The new pattern returns `Task` (void-like). The operation object is mutated in place.

6.11 PostgreSQL Connection String Configuration

Each module defines its own connection string name:

```
{
  "ConnectionStrings": {
    "Leasing": "",
    "Purchasing": "",
    "Vehicles": ""
  }
}
```

With development overrides in `appsettings.Development.json`:

```
{
  "ConnectionStrings": {
    "Leasing":
      "Host=postgres:5432;Database=leasebuy;Username=postgres;Password=mysecretpassword"
  }
}
```

Note the empty strings in `appsettings.json`. These are intentional — if a connection string is null or missing, `options.UseNpgsql(null)` throws a clear error at startup. This prevents silently connecting to the wrong database due to misconfiguration.

In Docker Compose, the host is `postgres` (the Docker service name). Locally, it would be `localhost`.

6.12 JetBrains.Annotations Usage

The `[UsedImplicitly]` attribute from JetBrains.Annotations is used to mark classes that are instantiated by frameworks (MediatR, MassTransit, ASP.NET Core) rather than explicitly by our code:

```
[UsedImplicitly]
internal sealed class PrepareLeaseCommandHandler :
    IRequestHandler<PrepareLeaseCommand, Guid>
{
}
```

Why this matters: Without `[UsedImplicitly]`, ReSharper and Rider would show "unused class" warnings for these classes. The attribute suppresses those warnings and documents that the class is used by the DI container/framework.

In the host project's `Program.cs` :

```
namespace EvolutionaryArchitecture.LeaseBuyArch
{
    [UsedImplicitly]
    public sealed class Program { }
}
```

This marks the `Program` class as used — it's the application entry point, used by `dotnet run` and by `WebApplicationFactory<T>` in integration tests.

6.13 FluentValidation Integration Details

The `FluentValidation.DependencyInjectionExtensions` package provides automatic validator registration in the DI container:

```
services.AddValidatorsFromAssembly(Assembly.GetExecutingAssembly());
```

This scans the assembly for all `Validator<T>` implementations and registers them. The `ValidateRequest<T>()` endpoint filter then resolves the validator and runs it before the handler:

```
// Conceptual implementation:
public static RouteHandlerBuilder ValidateRequest<TRequest>(this RouteHandlerBuilder
builder)
    where TRequest : class
{
    return builder.AddEndpointFilter(async (context, next) =>
    {
        var request = context.Arguments.OfType<TRequest>().FirstOrDefault();
        if (request is null) return await next(context);

        var validator = context.HttpContext.RequestServices
            .GetRequiredService<Validator<TRequest>>();
        var result = await validator.ValidateAsync(request);

        if (!result.IsValid)
            return Results.ValidationProblem(result.ToDictionary());

        return await next(context);
    });
}
```

Key packages: `FluentValidation` v11.7.1 and `FluentValidation.DependencyInjectionExtensions` v11.7.1 were used. These versions were the latest stable at the time and are compatible with .NET 10.

6.14 Port Allocation for Local Development

Each service listens on a different port for local development:

Chapter	Service	Port
---------	---------	------

Ch1	Monolith (LeaseBuyArch)	5000 (default)
Ch2	Modular Monolith	5000 (default)
Ch3	Leasing Microservice	5015
Ch3	Modular Monolith	5000 (default)

Why 5015 for Leasing? It's arbitrary but unique. When running both Ch3 services locally, they must listen on different ports. Port 5015 avoids conflicting with the monolith on port 5000.

In Docker Compose, ports are remapped:

- Modular Monolith: `8080:80` (host 8080 → container 80)
- Leasing Microservice: `8081:80` (host 8081 → container 80)
- PostgreSQL: `5432:5432`
- RabbitMQ Management: `15672:15672`
- RabbitMQ AMQP: `5672:5672`

6.15 The EndpointFilter vs. Middleware Trade-off

FluentValidation integration can be done two ways:

Approach 1: Endpoint Filter (used in Ch1)

```
app.MapPost("/api/leasing", async (...) => { ... })
    .ValidateRequest<PrepareLeaseRequest>();
```

- Per-endpoint — only validates specific endpoints
- No global impact
- Works with Minimal APIs

Approach 2: Global Middleware

```
app.UseMiddleware<ValidationMiddleware>();
```

- Validates ALL requests — can't skip specific endpoints
- Requires extracting request body twice (buffering)
- Better for global concerns (auth, logging)

Why Ch1 uses Approach 1: Only some endpoints need FluentValidation. The signing endpoint, for example, has an empty validator. Applying validation globally would add unnecessary overhead. Endpoint filters give granular control.

7. Architecture Decision Records

ADR-1: Project References vs. NuGet Packages for Common Layer

Context: The reference architecture publishes Common packages to GitHub Packages and references them via NuGet. This is the "right" way for microservices — each service depends on a versioned, independently releasable package.

Decision: Use project references instead of NuGet packages.

Rationale: NuGet packaging requires:

1. A packaging step (`dotnet pack`)
2. A versioning strategy (semantic versioning, changelog)
3. A package feed (GitHub Packages, NuGet.org, Azure Artifacts)
4. Authentication (for private feeds)
5. Package restore (each build fetches from the feed)

During development, project references provide instant feedback — change code in Common, rebuild, and it's reflected everywhere. There's no packaging step, no version bump, no feed configuration.

Trade-off: When you want to deploy the Leasing microservice independently from the monolith, you'll need to either:

1. Share the Common source code between both services (what we do now — works but couples deployments)
2. Publish Common as NuGet packages (correct but adds overhead)

When to revisit: Before production deployment of both services to different environments.

Approach for production: Create a CI/CD pipeline that:

1. Builds Common → publishes NuGet packages to an internal feed
2. Builds Leasing microservice (restores Common from feed)
3. Builds Modular Monolith (restores Common from feed)
4. Deploys each independently

ADR-2: In-Memory Event Bus vs. MassTransit

Context: Chapter 2 used an in-memory event bus backed by MediatR. Chapter 3 switched to MassTransit with RabbitMQ transport.

Decision: The in-memory bus is correct for a monolith. MassTransit is necessary when modules become separate processes.

Rationale: You should NOT add message broker complexity until you need it. The in-memory bus in Chapter 2:

- Has zero infrastructure dependencies (no RabbitMQ required)
- Has zero configuration (no host, username, password)
- Is trivially debuggable (events flow through MediatR pipelines)
- Is fast (in-process dispatch, no serialization overhead)

Chapter 3 introduces RabbitMQ because the Leasing module is now a separate service — in-process communication is no longer possible. The `IEventBus` interface remains the same; only the implementation changes.

Trade-off: MassTransit adds:

- Deployment complexity (RabbitMQ must be running)
- Configuration complexity (host, username, password)
- Debugging complexity (messages are opaque blobs in a queue)
- Performance overhead (serialization + network + deserialization)

Mitigation: The `IEventBus` interface stays the same (`PublishAsync<TEvent>`). The implementation can be swapped without changing any business logic code. If RabbitMQ is overkill, we could swap to an in-memory MassTransit transport for development.

ADR-3: Single Database Instance vs. Database-per-Service

Context: The reference architecture uses a single PostgreSQL database with separate schemas, even when the Contracts module is extracted as a microservice.

Decision: Keep a single database instance with schema separation.

Rationale: True database-per-service would require:

- Managing multiple PostgreSQL instances
- Configuring multiple connection strings
- Handling cross-service queries via API calls (instead of SQL joins)
- Managing data synchronization across databases

The schema separation provides sufficient isolation:

- Each service only connects to its own schema
- One service cannot accidentally read another service's tables
- Schemas can be backed up and restored independently
- Schema permissions can be granted per service

Trade-off: The database is a single point of failure. If PostgreSQL goes down, both services are affected. The database server must handle the combined load of both services.

When to revisit: When:

- The two services have different uptime requirements (e.g., Leasing needs 99.99% uptime, Comparison can tolerate 99%)
- The two services have different data storage needs (e.g., one needs TimescaleDB for time-series data)
- The single database instance becomes a performance bottleneck

ADR-4: Feature Flags via IConfiguration vs. IFeatureManager

Context: Chapter 2 used `IConfiguration.GetSection("FeatureManagement")` to check feature flags. Chapter 3's reference uses `IFeatureManager` from `Microsoft.FeatureManagement`.

Decision: Use `IFeatureManager` in Chapter 3 to match the reference architecture.

Rationale: `IFeatureManager` supports:

- **Percentage-based rollouts:** Enable a feature for 10% of requests
- **Time window filters:** Enable a feature only during business hours
- **Custom filters:** Enable a feature for specific users, regions, or A/B test groups
- **Dynamic updates:** Change flags at runtime without restarting the application (via Redis, Azure App Configuration, etc.)

For a simple on/off toggle, `IConfiguration` is sufficient and simpler. The reference pattern is more extensible and matches what a production system would use.

Trade-off: `IFeatureManager` requires:

- An additional NuGet package (`Microsoft.FeatureManagement`)
- An additional service registration (`AddFeatureManagement()`)
- A `BuildServiceProvider()` call during registration (code smell, as discussed)
- A `.Result` call on async methods (potential deadlock risk)

ADR-5: Module Record Location

Context: The `Module` record defines the module names used in feature flag checks and route registration. In Chapter 2, it existed in both `Common.Core` and the host project.

Decision: The `Module` record belongs in the **host project** (the composition root), not in `Common.Core`.

Rationale: The host project is the only place that needs to know about all modules — it's where modules are registered (in `Program.cs`) and where routes are mapped. Putting `Module` in `Common.Core`:

1. Creates a coupling where adding a module requires modifying `Common.Core`
2. Forces all modules to depend on a class with specific module names
3. Requires recompiling `Common.Core` when the module list changes

Trade-off: If you add a new module, you need to update:

1. The host project's `Module` record
2. The `Program.cs` registration
3. The `appsettings.json` feature flags

This is correct behavior — the host is the composition root, and the composition root is the only place that should know about all modules.

ADR-6: Three Solutions vs. One Solution

Context: Chapter 3 has three separate solution files (Common, Leasing, Monolith) instead of one unified solution.

Decision: Use three solutions, each with its own `Directory.Build.props`.

Rationale: In production, these would be separate repositories:

- Common → NuGet packages (published independently)
- Leasing → microservice (deployed independently)
- ModularMonolith → monolith (deployed independently)

During development, having separate solutions allows:

- Building only what you need (`make build-leasing` instead of building everything)
- Different target frameworks (not needed now, but possible)
- Different build configurations
- Easier CI/CD configuration (one solution = one build pipeline)

Trade-off: Project references between solutions work but are not visible in the IDE solution explorer. Cross-solution refactoring (e.g., renaming a class in Common used by Leasing) won't propagate automatically.

ADR-7: FluentValidation vs. Data Annotations

Context: The reference architecture uses FluentValidation for request validation rather than ASP.NET Core's built-in Data Annotations (`[Required]` , `[Range]` , etc.).

Decision: Use FluentValidation.

Rationale:

1. **Separation of concerns:** Validation rules are in separate classes, not embedded in request DTOs. The request record stays clean.
2. **Composability:** Validators can be combined, inherited, and reused.

3. **Conditional logic:** FluentValidation supports complex validation (`When` , `Unless` , `Must`) that would be awkward with Data Annotations.
4. **Automatic registration:** `FluentValidation.DependencyInjectionExtensions` auto-registers validators in DI via assembly scanning.
5. **Testability:** Validators can be unit tested independently from endpoints.

Trade-off: An additional NuGet dependency. Data Annotations would work without any packages.

Example of conditional validation only possible with FluentValidation:

```
RuleFor(request => request.DownPayment)
    .GreaterThan(0)
    .When(request => request.VehicleMsrp > 50000)
    .WithMessage("Vehicles over $50,000 require a down payment");
```

ADR-8: Endpoint Route Registration Pattern

Context: Each chapter organizes endpoint mapping differently:

- Ch1: Endpoint classes with `MapPost(...)` called from a central `MapXxx()` method
- Ch2/Ch3: `IEndpointRouteBuilder` extension methods in `LeasingEndpoints.cs`

Decision: Collect all endpoint mappings in a static `MapXxx()` method per module.

Rationale:

1. **Discoverability:** `LeasingEndpoints.MapLeasing()` shows ALL Leasing endpoints in one place.
2. **Conditional registration:** Feature flags gate endpoint registration:

```
public static WebApplication RegisterLeasing(this WebApplication app, string module)
{
    if (!app.IsModuleEnabled(module)) return app;
    app.RegisterLeasingInfrastructure();
    app.MapLeasing();
    return app;
}
```

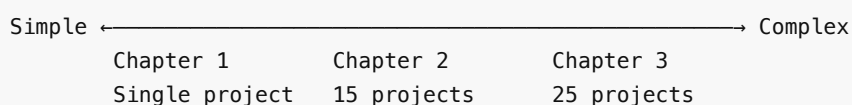
3. **Registration order:** Endpoints can be ordered, which matters for catch-all routes.

Trade-off: More files than putting everything in `Program.cs` , but the separation is worth it for any non-trivial application.

8. Epilogue — Lessons for Future Me

8.1 Architecture is a Process, Not a Destination

The three chapters don't represent "good, better, best." They represent **different points on a complexity axis**:



No DI	MediatR	MassTransit
No event bus	In-memory bus	RabbitMQ

The RIGHT architecture is the one that matches your current constraints:

- **Team of 1 building an MVP?** Start at Chapter 1.
- **Team of 5 building a product?** Start at Chapter 2.
- **Three teams of 5 building a platform?** Start at Chapter 3 (or even skip directly to services).

The key insight is that you can **evolve** from one chapter to the next without a rewrite. The `IEventBus` interface is the same in Chapter 2 and Chapter 3. The module registration pattern is the same. The business logic in Core is the same. Only the infrastructure changes.

8.2 What I Would Do Differently

1. **Write integration tests in Chapter 1.** Without tests, refactoring from monolith to modular monolith was terrifying. I had to manually verify every endpoint. Integration tests would have given me confidence that the refactoring didn't break anything.
2. **Use the same .NET version from the start.** Jumping between .NET 7 (reference) and .NET 10 (our implementation) caused compatibility issues with package versions, API changes (OpenAPI transformers), and build tooling (`.slnx` format).
3. **Add OpenTelemetry from Chapter 2.** Observability is critical for any non-trivial application, regardless of architecture. When events flow through MediatR or MassTransit, you need distributed tracing to understand what's happening.
4. **Don't fight the framework.** .NET 10 has strong opinions about nullable reference types, treat-warnings-as-errors, and package pruning. Fighting these by adding `NoWarn` everywhere is a losing battle. Embrace the conventions.
5. **Add a GlobalUsings.cs file for each project.** The shared `Directory.Build.props` global usings work for most projects, but some projects need additional usings. A `GlobalUsings.cs` file is a cleaner approach than adding more `Using` elements to `Directory.Build.props`.
6. **Document the build order dependencies.** The fact that `AddFeatureManagement()` must be called before `AddCommonInfrastructure()` is not obvious. It should be documented in comments and in the architecture docs.

8.3 The Complete Technology Checklist

If I were starting this project from scratch today:

- ☐ `dotnet new webapi` — initial project scaffold
- ☐ Add `Directory.Build.props` — shared build configuration
- ☐ Add EF Core + Npgsql — database access
- ☐ Add `docker-compose.yml` — PostgreSQL + RabbitMQ
- ☐ Add MediatR — in-process messaging (Chapter 2+)
- ☐ Add MassTransit + RabbitMQ — out-of-process messaging (Chapter 3+)
- ☐ Add `Microsoft.FeatureManagement` — feature flags (Chapter 2+)
- ☐ Add Swashbuckle + `Microsoft.AspNetCore.OpenApi` — API documentation
- ☐ Add Dapper — read-only queries (Comparison module)

- ☐ Add JetBrains.Annotations — [UsedImplicitly] attribute
- ☐ Add Makefile — build orchestration

8.4 The Mental Model

Chapter 1: "Get it working."

No patterns. No abstraction. Just code.

Chapter 2: "Keep it working as it grows."

Add structure. Add boundaries. Add events.

Chapter 3: "Split it when one part needs to scale independently."

Extract modules. Add async communication. Accept operational complexity.

Each chapter is a response to a **pain point**:

- **Chapter 1 pain:** "I don't understand the domain well enough to structure it."
- **Chapter 2 pain:** "The monolith is becoming hard to navigate and change safely."
- **Chapter 3 pain:** "The Leasing module has different deployment needs than the rest."

Don't introduce Chapter 2 patterns in Chapter 1. Don't introduce Chapter 3 patterns in Chapter 2.

Each evolutionary step is driven by necessity, not by architectural fashion.

8.5 Key Numbers

| Metric | Chapter 1 | Chapter 2 | Chapter 3 | |---|---|---|---|---| | Projects | 1 | 15 | 25 | | Solutions | 1 | 1 | 3 | | Source files | ~25 | ~80 | ~110 | | Build time (clean) | ~5s | ~15s | ~25s | | NuGet packages | ~7 | ~12 | ~16 | | Docker services | 2 (app + db) | 2 (app + db) | 4 (2 apps + db + mq) | | Event bus | None | In-memory (MediatR) | Out-of-process (MassTransit) | | Feature flags | None | IConfiguration | IFeatureManager | | Validation | FluentValidation | FluentValidation | FluentValidation | | API docs | Swashbuckle + OpenAPI | Swashbuckle + OpenAPI | Swashbuckle + OpenAPI | | ORM | EF Core + Npgsql | EF Core + Npgsql + Dapper | EF Core + Npgsql + Dapper | | Messaging | None | MediatR | MassTransit + RabbitMQ | | Solution format | .sln | .slnx | .slnx (mixed with .sln) |

8.6 Final Words

Evolutionary architecture is not about designing the perfect system upfront. It's about making **incremental, reversible decisions** that keep your options open. A monolith is reversible (you can modularize it later). A microservice is not easily reversible (merging services is harder than splitting them).

Start simple. Add complexity only when the pain of NOT having it exceeds the pain of having it. And document your journey so future you knows why decisions were made.

8.7 Recommended Future Work

If I continue this project, here's what I'd tackle next:

1. **Add the test suite** described in Section 9. Without tests, the refactoring from Ch1→Ch2 was manual and error-prone.
2. **Publish Common as NuGet packages** and consume them from both services. This enables truly independent deployment.
3. **Add OpenTelemetry** with distributed tracing across RabbitMQ. When a LeaseSignedEvent flows from the Leasing service to the Modular Monolith, you want to see the entire trace in one view.

4. **Add API versioning** (e.g., `/api/v1/leasing`). The `ApiPaths.Root` constant makes this a one-line change.
5. **Add health check endpoints** for both services, checking PostgreSQL and RabbitMQ connectivity.
6. **Switch to database-per-service** when the Leasing microservice needs its own PostgreSQL instance.
7. **Add authentication/authorization** (the current app has none — anyone can prepare or sign a lease).
8. **Containerize the development environment** with `docker compose up` for a one-command developer setup.

8.8 Final Build Verification

After all three chapters, this is how you verify everything builds:

```
export PATH="/usr/local/share/dotnet:$PATH"

# Chapter 1
cd leasebuyarch && make clean && make build
# Build succeeded. 0 Warning(s) 0 Error(s)

# Chapter 2
cd leasebuyarch2 && make clean && make build
# Build succeeded. 0 Warning(s) 0 Error(s)

# Chapter 3
cd leasebuyarch3 && make clean && make build
# Build succeeded. 0 Warning(s) 0 Error(s) - 3 times (Common, Leasing, Monolith)
```

9. Testing Strategy

9.1 Current State

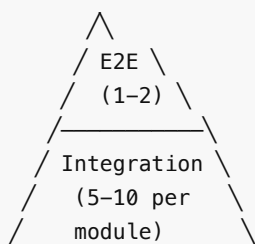
As of this writing, there are **no automated tests** in any of the three chapters. The `InternalsVisibleTo` directive in `Directory.Build.props` hints at test projects:

```
<InternalsVisibleTo Include="DynamicProxyGenAssembly2" />
```

This enables Moq to mock internal types, which is necessary because all domain classes (Lease, Purchase, Vehicle) are `internal sealed class`.

9.2 Recommended Test Structure

For a production version of this application, the testing pyramid would look like:



Unit Tests
(10–20 per module)

9.2.1 Unit Tests (Foundation)

What to test: Business rules, calculation formulas, entity behavior.

Example test for the leasing calculation:

```
public sealed class LeaseCalculationTests
{
    [Fact]
    public void Calculate_monthly_payment_for_standard_lease()
    {
        var msrp = 45000m;
        var residualPercentage = 55m;
        var moneyFactor = 0.00125m;
        var termMonths = 36;

        var lease = Lease.Prepare(
            Guid.NewGuid(), Guid.NewGuid(), msrp,
            residualPercentage, moneyFactor, termMonths,
            12000, 750, DateTimeOffset.UtcNow);

        Assert.Equal(649.69m, lease.MonthlyPayment);
    }

    [Fact]
    public void Credit_score_below_700_throws_business_rule_exception()
    {
        Assert.Throws<BusinessRuleValidationException>(() =>
            Lease.Prepare(Guid.NewGuid(), Guid.NewGuid(), 45000m, 55m, 0.00125m, 36,
            12000, 699, DateTimeOffset.UtcNow));
    }

    [Fact]
    public void Signing_after_14_days_throws_business_rule_exception()
    {
        var preparedAt = new DateTimeOffset(2026, 1, 1, 0, 0, 0, TimeSpan.Zero);
        var lease = Lease.Prepare(Guid.NewGuid(), Guid.NewGuid(), 45000m, 55m,
            0.00125m, 36, 12000, 750, preparedAt);
        var signedAt = preparedAt.AddDays(15);
        Assert.Throws<BusinessRuleValidationException>(() => lease.Sign(signedAt));
    }
}
```

What makes a good unit test: Tests ONE behavior, uses `ISystemClock` fake (not `DateTimeOffset.UtcNow`), tests boundary conditions (credit score exactly 700, mileage exactly 15,000, term exactly 36 months), and tests edge cases (score 699 fails, score 700 passes).

9.2.2 Integration Tests (Middle Layer)

What to test: Endpoint behavior, database interaction, event publishing. Use `WebApplicationFactory<Program>` :

```
public class LeaseEndpointTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;

    public LeaseEndpointTests(WebApplicationFactory<Program> factory)
    {
        _factory = factory.WithWebHostBuilder(builder =>
        {
            builder.ConfigureTestServices(services =>
            {
                services.RemoveDbContext<LeasingPersistence>();
                services.AddDbContext<LeasingPersistence>(options =>
                    options.UseInMemoryDatabase("Test"));
            });
        });
    }

    [Fact]
    public async Task POST_leasing_returns_201_with_valid_request()
    {
        var client = _factory.CreateClient();
        var request = new PrepareLeaseRequest(
            Guid.NewGuid(), Guid.NewGuid(), 45000m, 55m, 0.00125m, 36, 12000, 750);
        var response = await client.PostAsJsonAsync("/api/leasing", request);
        Assert.Equal(HttpStatusCode.Created, response.StatusCode);
    }

    [Fact]
    public async Task POST_leasing_with_low_credit_score_returns_409()
    {
        var client = _factory.CreateClient();
        var request = new PrepareLeaseRequest(
            Guid.NewGuid(), Guid.NewGuid(), 45000m, 55m, 0.00125m, 36, 12000, 650);
        var response = await client.PostAsJsonAsync("/api/leasing", request);
        Assert.Equal(HttpStatusCode.Conflict, response.StatusCode);
    }
}
```

What to test: HTTP status codes (201, 204, 404, 409), business rule violations return 409 (not 500), events are published, database state is persisted.

9.2.3 E2E Tests (Apex)

Complete user journeys across modules. Example Gherkin scenario:

```
Scenario: Customer leases a vehicle
    Given a vehicle exists with MSRP $45,000
```

```
When the customer prepares a lease with valid parameters
Then the lease is created with a calculated monthly payment
When the customer signs the lease within 14 days
Then the vehicle status changes to "Leased"
```

9.3 Test Doubles Strategy

Dependency	Unit Test	Integration Test	E2E Test
ISystemClock	Fake (fixed date)	Fake (fixed date)	Real
IEventBus	Mock	Fake (in-memory)	Real (RabbitMQ)
Database	None	InMemory or Testcontainers	Real (Docker)

9.4 The InternalsVisibleTo Directive

Without `InternalsVisibleTo`, Moq cannot mock `internal sealed` classes. Since all domain entities are `internal sealed`, this is essential for testing. Alternative: make domain types `public`, but the reference architecture uses `internal` to enforce encapsulation.

9.5 Testing the Event Bus Transition

Three approaches for testing cross-process messaging:

1. **MassTransit in-memory transport:** Replace RabbitMQ in tests:

```
configurator.UsingInMemory((context, cfg) =>
    cfg.ConfigureEndpoints(context));
```

2. **Testcontainers:** Use `Testcontainers.RabbitMq` to spin up a real RabbitMQ container.

3. **IEventBus substitution:** Swap in a test implementation:

```
internal sealed class TestEventBus : IEventBus
{
    public List<IIntegrationEvent> PublishedEvents { get; } = new();
    public Task PublishAsync<TEvent>(TEvent @event, CancellationToken ct)
        where TEvent : IIntegrationEvent
    {
        PublishedEvents.Add(@event);
        return Task.CompletedTask;
    }
}
```